

ELECTRONIC PRINCIPLES AND DEVICES

Course Content

Department of Electronics and Communication.

ELECTRONIC PRINCIPLES AND DEVICES

Unit 4 –DIGITAL ELECTRONICS

[https://
www.youtube.com/
playlist?
list=PLwjK_iyK4LLBC_s
o3odA64E2MLgIRKafl](https://www.youtube.com/playlist?list=PLwjK_iyK4LLBC_so3odA64E2MLgIRKafl)

Digi-Elec yt Playlist

ELECTRONIC PRINCIPLES AND DEVICES

Introduction

Digital electronics is a branch that deals with circuits and systems using digital signals.

Works with discrete signals represented by binary values— 0 and 1.

These binary values correspond to two voltage levels, typically representing "low" (0) and "high" (1).

Number System

Each number system is characterized by its **base** or **radix**, which determines the number of symbols used.

Base (or **Radix**) refers to the number of unique digits or symbols used in a number system.

For example: The **decimal system** has a base of 10, using digits 0 to 9.

The **binary system** has a base of 2, using digits 0 and 1.

The **hexadecimal system** has a base of 16, using digits 0 to 9 and letters A to F.

ELECTRONIC PRINCIPLES AND DEVICES

Binary Number System

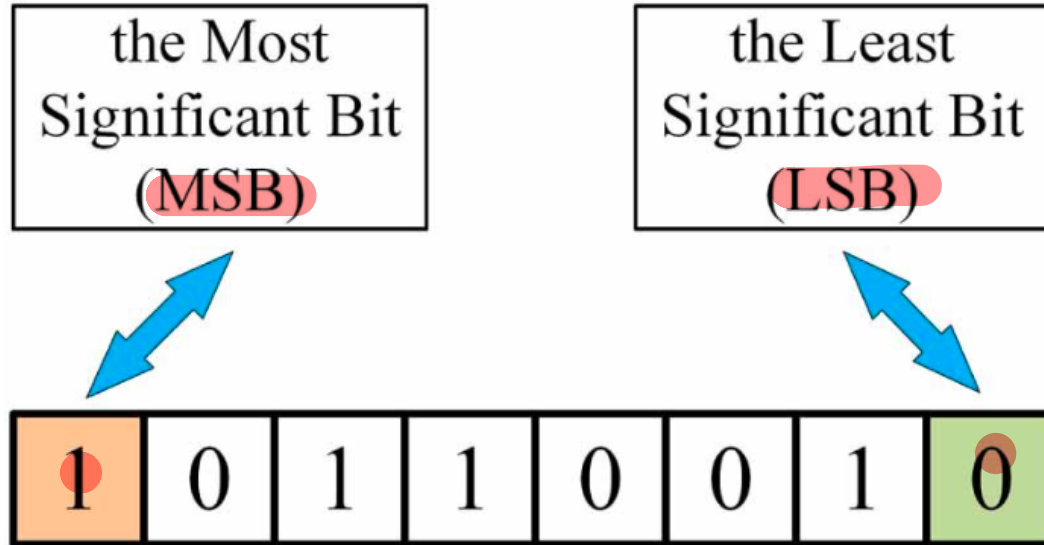
Key features

- The **binary number system** is a **base-2** system that uses only two digits: **0** and **1**.
- Each digit in a binary number is called a **bit** (short for "binary digit").
- **Place Value:** Each bit in a binary number represents a specific power of 2, starting from 2^0 (rightmost bit) and increasing to $2^1, 2^2, 2^3$, etc., as you move leftward.
- **Representation of Numbers:**
Any numerical value can be represented in binary form using combinations of 0s and 1s.
The number of **bits** in a binary number determines the total number of unique combinations or values that can be represented.

If n is the number of bits, the maximum number values that can be represented using n bits equals 2^n

ELECTRONIC PRINCIPLES AND DEVICES

- In a binary number, the rightmost bit is called the **Least Significant Bit (LSB)** and the left most bit is the **Most Significant Bit (MSB)**.



The place value of the LSB is 2^0 and the next digits to the left get the place value in terms of increasing powers of 2. In the above number, the place value of MSB is 2^7 .

ELECTRONIC PRINCIPLES AND DEVICES

- Range of Values for Unsigned Binary Numbers

In **unsigned binary**, all values are non-negative, starting from 0.

$$\text{Range} = 0 \text{ to } (2^n - 1)$$

For ex: With $n=1$ bit: no of values is $2^1=2$ values (0,1)

With $n=2$ bits: no of values is $2^2=4$ values (00,01,10,11), 0 to 3 in decimal.

With $n=3$ bits: $2^3=8$ values (000-111), 0 to 7 in decimal

- Range of Values for Signed Binary Numbers:

In **signed binary**, one bit is reserved as the **sign bit** (0 for positive, 1 for negative).

$$\text{Range} = -2^{n-1} \text{ to } (2^{n-1} - 1)$$

For ex: With **2 bits**: $2^2=4$ values (-2 to 1), -2, -1, 0, 1

With **3 bits**: $2^3=8$ values (-4 to 3), -4, -3, -2, -1, 0, 1, 2, 3

ELECTRONIC PRINCIPLES AND DEVICES

2 bit binary

Representation	Range
00	0
01	1
10	2
11	3

3 bit binary

Representation	Range
000	0
001	1
010	2
011	3
100	4
101	5
110	6
111	7

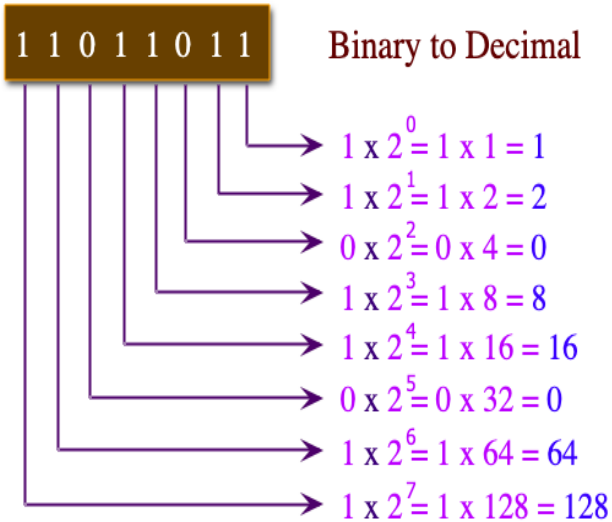
4 bit binary

Representation	Range
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	10
1011	11
1100	12
1101	13
1110	14
1111	15

ELECTRONIC PRINCIPLES AND DEVICES

Binary to Decimal Conversion:

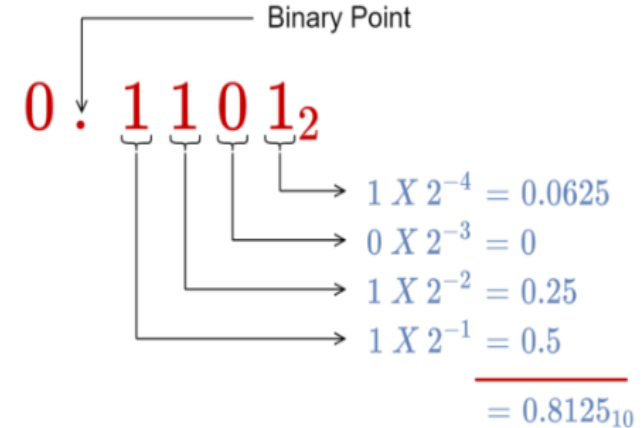
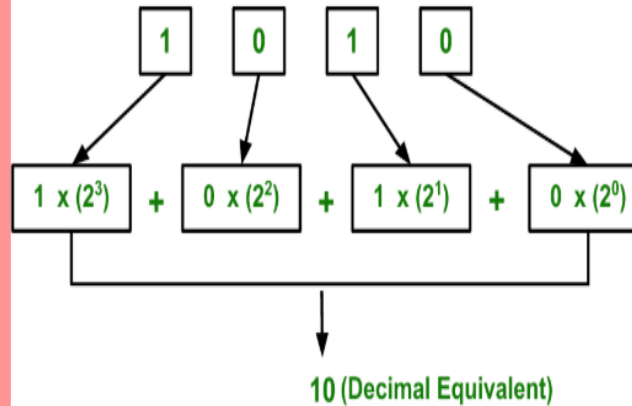
- Multiply each binary digit by the value of its position or place value (power of 2).
- Add all the products together to get the decimal equivalent



$$1 + 2 + 8 + 16 + 64 + 128 = 219$$

$$(11011011)_2 = (219)_{10}$$

Binary number - 1010



Convert the following binary numbers to its decimal equivalent

a) 1011

Starting from LSB,

$$1 \times 2^0 + 1 \times 2^1 + 0 \times 2^2 + 1 \times 2^3 \\ = 1 + 2 + 0 + 8 = 11$$

$$(1011)_2 = (11)_{10}$$

b) 11010

$$(11010)_2 = (26)_{10}$$

c) 1111101

$$(1111101)_2 = (127)_{10}$$

d) 101.01011

$$101_2 = (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) \\ = (1 \times 4) + (0 \times 2) + (1 \times 1) \\ = 4 + 0 + 1 = 5$$

$$.01011_2 = (0 \times 2^{-1}) + (1 \times 2^{-2}) + (0 \times 2^{-3}) + (1 \times 2^{-4}) + (1 \times 2^{-5}) \\ = 0 + \frac{1}{4} + 0 + \frac{1}{16} + \frac{1}{32} \\ = 0 + 0.25 + 0 + 0.0625 + 0.03125 = 0.34375$$

$$101.01011_2 = 5.34375_{10}$$

ELECTRONIC PRINCIPLES AND DEVICES

Decimal to Binary Conversion

- Divide the Decimal Number by 2.
- Record the quotient and the remainder.
- Repeat the division by 2 until the quotient is 0.
- The last remainder represents the MSB and the first remainder represents the LSB. The remainders represent the equivalent binary number

2	4215		
2	2107	— 1	← LSB
2	1053	— 1	
2	526	— 1	
2	263	— 0	
2	131	— 1	
2	65	— 1	
2	32	— 1	
2	16	— 0	
2	8	— 0	
2	4	— 0	
2	2	— 0	
2	1	— 0	
	0	— 1	← MSB

Decimal to Binary Conversion

$$(27)_{10} = (11011)_2$$

2	27	Remainder
2	13	1
2	6	1
2	3	0
2	1	1
	0	1

Convert $(0.6875)_{10}$ to binary.

	Integer		Fraction
$0.6875 \times 2 =$	1	+	0.3750
$0.3750 \times 2 =$	0	+	0.7500
$0.7500 \times 2 =$	1	+	0.5000
$0.5000 \times 2 =$	1	+	0.0000

$$(0.6875)_{10} = (0.1011)_2$$

Convert the decimal number 41 to binary.

	Integer Quotient		Remainder	Coefficient
$41/2 =$	20	+	$\frac{1}{2}$	$a_0 = 1$
$20/2 =$	10	+	0	$a_1 = 0$
$10/2 =$	5	+	0	$a_2 = 0$
$5/2 =$	2	+	$\frac{1}{2}$	$a_3 = 1$
$2/2 =$	1	+	0	$a_4 = 0$
$1/2 =$	0	+	$\frac{1}{2}$	$a_5 = 1$

$$(41)_{10} = (101001)_2$$

ELECTRONIC PRINCIPLES AND DEVICES

Hexadecimal Number System

The **hexadecimal (base-16)** number system is a positional numeral system that uses **16 distinct symbols** to represent values.

Symbols Used:

- **Digits:** 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- **Letters:** A, B, C, D, E, F
(where A = 10, B = 11, C = 12, D = 13, E = 14, F = 15 in decimal).

•Place Values:

Each digit in a hexadecimal number represents a power of 16, starting from 16^0 at the rightmost position

Ex: 1A316, 2AC

Hexadecimal	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Decimal	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

ELECTRONIC PRINCIPLES AND DEVICES

Hexadecimal to Decimal Conversion

- Convert each hexadecimal bit into its decimal equivalent.
- Multiply each bit by the value of its position or place value (power of 16).
- Add all the products together to get the decimal equivalent

2F36

$$6 \times 16^0 + 3 \times 16^1 + 15 \times 16^2 + 2 \times 16^3$$

$$= 8192 + 3840 + 48 + 6$$

$$= 12086$$

Hexadecimal	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Decimal	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

$(B65F)_{16}$

$$(B65F)_{16} = 11 \times 16^3 + 6 \times 16^2 + 5 \times 16^1 + 15 \times 16^0 = (46,687)_{10}$$

ELECTRONIC PRINCIPLES AND DEVICES

Decimal to Hexadecimal Conversion

Divide the Decimal Number by 16

Divide the decimal number by 16. The quotient will be used for the next division, and the remainder will be one of the hexadecimal digits.

Record the Remainder

The remainder of each division corresponds to a hexadecimal digit. If the remainder is between 10 and 15, use the hexadecimal letters:

Hexadecimal	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Decimal	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Repeat the Division

Divide the quotient by 16 again, and continue recording the remainders until the quotient is 0.

Read the Remainders in Reverse Order

The remainders, when read from last to first, form the hexadecimal representation of the decimal number.

Example 1: Convert 255 (Decimal) to Hexadecimal

Step 1: Divide 255 by 16:

$$255 \div 16 = 15 \text{ (quotient)} \quad \text{remainder} = 15$$

Step 2: Divide the quotient (15) by 16:

$$15 \div 16 = 0 \text{ (quotient)} \quad \text{remainder} = 15$$

Step 3: The quotient is now 0, so stop dividing.

Step 4: Read the remainders in reverse order: The remainders, from last to first, are F and F.
Thus, **255 (decimal) = FF (hexadecimal).**

ELECTRONIC PRINCIPLES AND DEVICES

1's Complement



One's Complement

If all bits in a byte are inverted by changing each 1 to 0 and each 0 to 1, we have formed the one's complement of the number.

Original	One's Complement
----------	------------------

10011001	--> 01100110
----------	--------------

10000001	--> 01111110
----------	--------------

11110000	--> 00001111
----------	--------------

11111111	--> 00000000
----------	--------------

00000000	--> 11111111
----------	--------------

1's Complement Representation in Signed Magnitude :

In sign-magnitude, 6 is a positive number, its binary representation is 0000 0110, and it will be represented as it is, i.e. 0000 0110. Whereas a negative number suppose -6 will be represented as 1111 1001

ELECTRONIC PRINCIPLES AND DEVICES

2's Complement

To form the two's complement, add 1 to the one's complement.

Step 1: Begin with the original binary value

10011001 Original byte

Step 2: Find the one's complement

01100110 One's complement

Step 3: Add 1 to the one's complement

```
01100110 One's complement
+      1 Add 1
-----
01100111 <--- Two's complement
```

ELECTRONIC PRINCIPLES AND DEVICES

2's Complement

2's Complement Representation in Signed Magnitude :

For signed binary numbers the most significant bit (MSB) is used as the sign bit.

If the sign bit is “0”, this means the number is positive in value.

If the sign bit is “1”, then the number is negative in value.

1. **2's Complement** is a method used to represent signed integers (both positive and negative numbers) in binary.

- **Positive numbers:** Their 2's complement is the same as their standard binary representation.
- **Negative numbers:** Their 2's complement is obtained by taking the binary representation of the positive value, inverting the bits (1's complement), and adding 1.

ELECTRONIC PRINCIPLES AND DEVICES

Binary Addition using 2's Complement

(1): $8 + 5 = ?$

2's complement representation of 8 = 00001000

2's complement representation of 5 = 00000101

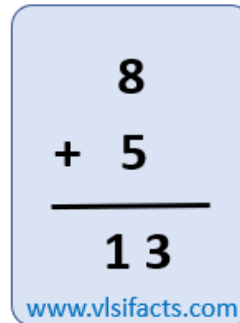
Since 8 and 5 are positive, so their 2's complement representation will be the same as its true (uncomplemented) form.

$$\begin{array}{r} 00001000 \\ + 00000101 \\ \hline 00001101 \end{array}$$

Sum: 00001101



Sign bit is zero, so the result is a positive number


$$\begin{array}{r} 8 \\ + 5 \\ \hline 13 \end{array}$$

www.vlsifacts.com

13 => 00001101

ELECTRONIC PRINCIPLES AND DEVICES

Binary Addition using 2's Complement

(2) $8 + (-5) = ?$

This example can also be read as $8 - 5 = ?$

2's complement representation of 8 = 00001000

2's complement representation of -5 = 11111011

$$\begin{array}{r} 00001000 \\ + 11111011 \\ \hline 100000011 \end{array}$$

Discard Carry 1

Sum: 00000011

↑
Sign bit is zero, so the result is a positive number

$$\begin{array}{r} 8 \\ + -5 \\ \hline 3 \end{array}$$

www.vlsifacts.com

3 => 00000011

ELECTRONIC PRINCIPLES AND DEVICES

Binary Subtraction using 2's Complement

(1) Subtract $(1010)_2$ from $(1111)_2$ using 2's complement method.

Solution:

Step-1: 2's complement of $(1010)_2$ is $(0110)_2$.

Step-2: Add $(0110)_2$ to $(1111)_2$. This is shown below.

method of 2s complement subtraction of smaller number from a larger number

$$\begin{array}{r} 1\ 1\ 1\ 1 \\ + 0\ 1\ 1\ 0 \\ \hline \text{Omit this carry } \textcircled{1}\ 0\ 1\ 0\ 1 \\ \downarrow \\ \boxed{0\ 1\ 0\ 1} \text{ Answer} \end{array}$$

ELECTRONIC PRINCIPLES AND DEVICES

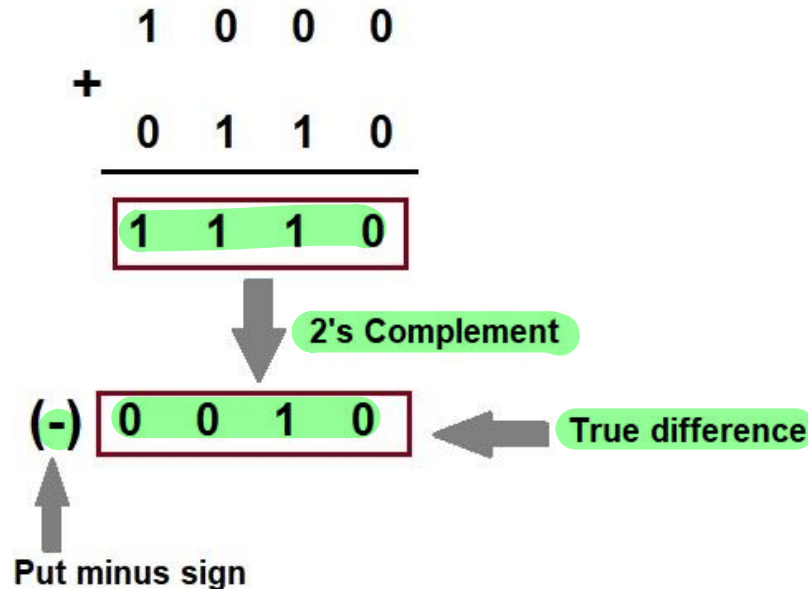
Binary Subtraction using 2's Complement

(1) **Example-2:** Subtract $(1010)_2$ from $(1000)_2$ using 2's complement.

Solution:

Step-1: Find the 2's complement of $(1010)_2$. It is $(0110)_2$.

Step-2: Add $(0110)_2$ to $(1000)_2$.



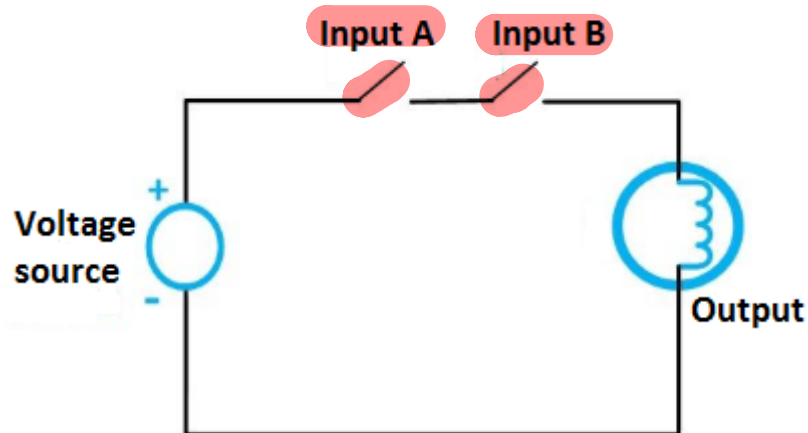
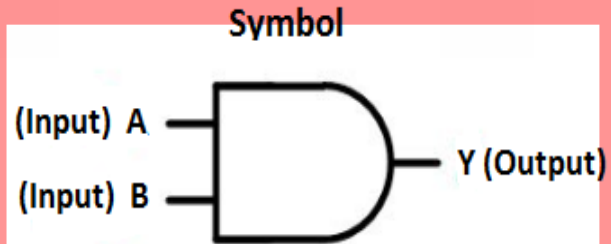
- ❖ Digital Electronic circuits process data that contains **binary values 1's and 0's**.
- ❖ A **Logic "1"** is also referred as HIGH voltage or TRUE or ON state
- ❖ A **Logic "0"** is also referred as LOW voltage or FALSE or OFF state
- ❖ A binary digit "0" or "1" is called a **bit**
- ❖ Digital information is stored using a series of binary values 1's and 0's.
- ❖ Digital systems such as digital telephones, digital cameras, computers, handheld portable devices and other high technology systems **stores and process** these binary values (1's and 0's)

Boolean Algebra and Logic gates

- ❖ Boolean Algebra is the mathematics used to analyse and **simplify logic or digital circuits**.
- ❖ Digital circuits are constructed by using **logic gates**.
- ❖ Logic gates are the **basic building blocks of digital systems, performs logical functions based on Boolean algebra**.
- ❖ Logic gates have **one or more inputs and only one output**.
- ❖ Number of possible input states is **2^n** . Where n is Number of inputs
- ❖ Logic gates are implemented using diodes or transistors which acts as **electronic switches**
- ❖ logic gates : (i) **Basic Gates: AND, OR, NOT,**
(ii) **Derived Gates: NAND, NOR, EXOR and EXNOR gates**

Boolean Algebra and Logic gates

- ❖ **AND Gate** is an electronic circuit that gives output as logic “1” (HIGH) only if all inputs are “1”. Otherwise output is Logic “0” (LOW)
- ❖ Boolean algebra representation for AND gate is $Y = A.B$
- ❖ AND gate is represented as **series connection** of two switches

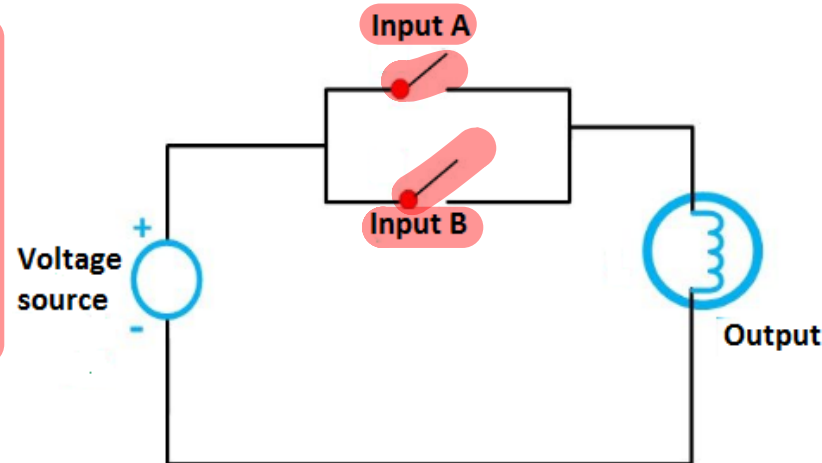
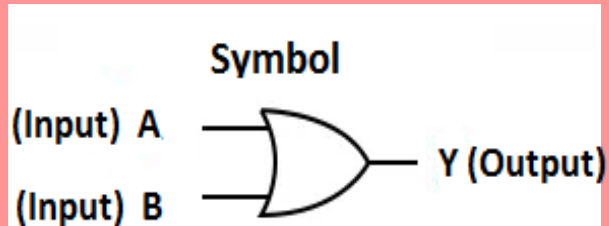


Truth Table

Inputs		Output
A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

Boolean Algebra and Logic gates

- ❖ **OR Gate** is an electronic circuit that gives as Logic “1” (HIGH) if any of the inputs are HIGH .Output of OR gate goes Logic “0” (LOW) only if all inputs are Logic “0” (LOW)
- ❖ Boolean algebra representation of OR gate is $Y = A + B$
- ❖ OR gate is represented as **parallel connection** of two switches



Truth Table

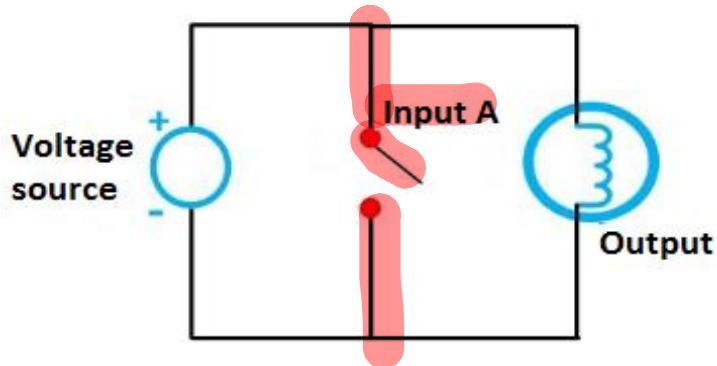
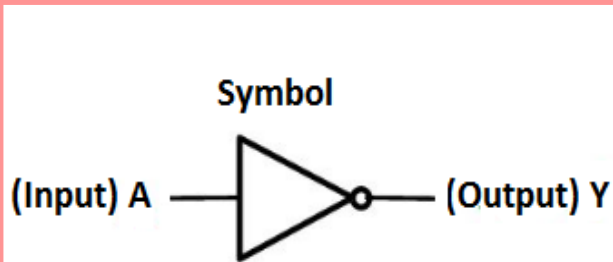
Inputs		Output
A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

Boolean Algebra and Logic gates

❖ **NOT Gate** is a single input and single output gate which performs the inversion of the applied binary input signal. Hence it is also called as **Inverter Gate**.

❖ Boolean expression for NOT gate is $Y = \bar{A}$

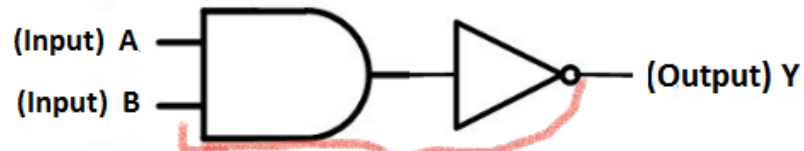
❖ NOT Gate can be represented by connecting a switch parallel to bulb



Truth Table

Input	Output
A	Y
0	1
1	0

NAND Gate $1\ 1 = 0$



Symbol

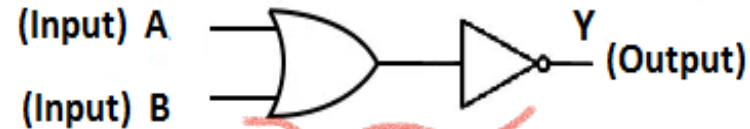


Truth Table

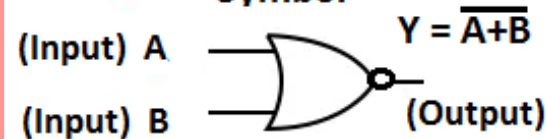
NAND		
Input		Output
A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

12/14/2024

NOR Gate $0\ 0 = 1$



Symbol



Truth Table

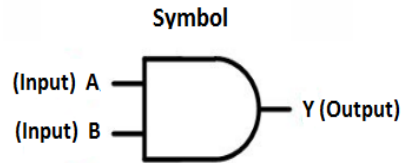
NOR		
Input		Output
A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

Boolean Algebra and Logic gates

❖ Logical Boolean Laws:

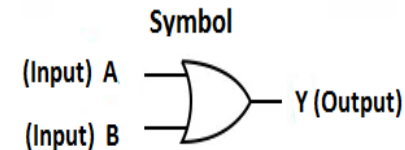
- ❖ **NOT / Inversion Law:** $\overline{1} = 0$ (i) If $A = 1$ then $A' = 0$ and if $A = 0$ then $A' = 1$
 $\overline{0} = 1$ (ii) $(A')' = A$ itself

❖ **AND Law:**



$0.0 = 0$	$A.0 = 0$
$0.1 = 0$	$A.1 = A$
$1.0 = 0$	$A.A = A$
$1.1 = 1$	$A.A' = 0$

❖ **OR Law:**



$0+0 = 0$	$A+0 = A$
$0+1 = 1$	$A+1 = 1$
$1+0 = 1$	$A+A = A$
$1+1 = 1$	$A+A' = 1$

Boolean Algebra and Logic gates

❖ Principle of duality in Boolean Algebra

❖ In Boolean Algebra , One type of expression can be converted into another type of expression by replacing “0 with 1 “ , “1 with 0” , “(+) sign with (.) sign “ and vice versa.

Example:

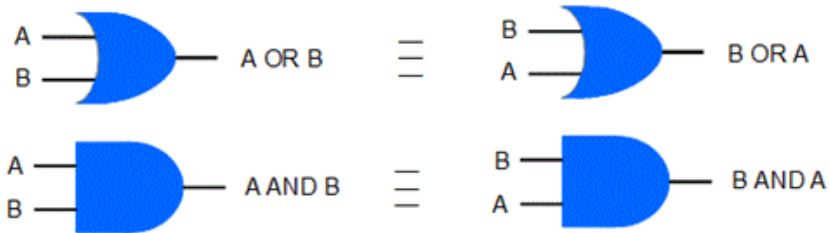
AND (.)	OR (+)
$0.0 = 0$	$1+1 = 1$
$0.1 = 0$	$1+0 = 1$
$1.0 = 0$	$0+1 = 1$
$1.1 = 1$	$0+0 = 0$

Expression	Dual Expression
$\overline{0} = 1$	$\overline{1} = 0$
$0.1 = 0$	$1 + 0 = 1$
$A.0 = 0$	$A + 1 = 1$
$A.B = B.A$	$A + B = B + A$
$A.\overline{A} = 0$	$A + \overline{A} = 1$

Basic Theorem and Properties of Boolean Algebra

❖ Boolean Laws and Theorems are used to simplify the Boolean expressions.
Hence reduce the number of logic gates.

❖ Commutative Laws: (i) $A + B = B + A$
(ii) $A \cdot B = B \cdot A$



Proof: Logic circuits

A	B	(A+B)	(B+A)	(A.B)	(B.A)
0	0	0	0	0	0
0	1	1	1	0	0
1	0	1	1	0	0
1	1	1	1	1	1

Proof: Truth Table

Basic Theorems and Properties of Boolean Algebra

❖ Associative Law:

$$(A+B)+C = A+(B+C) \quad \text{Boolean addition}$$

❖ Proof: Truth Table

A	B	C	A + B	(A + B) + C	B + C	A + (B + C)
0	0	0	0	0	0	0
0	0	1	0	1	1	1
0	1	0	1	1	1	1
0	1	1	1	1	1	1
1	0	0	1	1	0	1
1	0	1	1	1	1	1
1	1	0	1	1	1	1
1	1	1	1	1	1	1

❖ From principle of duality: $(A.B).C = A.(B.C)$ Boolean Multiplication

Basic Theorem and Properties of Boolean Algebra

❖ Distributive Law:

$$A + B.C = (A+B) . (A+C)$$

❖ Proof: Truth Table

A	B	C	BC	A+BC	(A+B)	(A+C)	(A+B)(A+C)
0	0	0	0	0	0	0	0
0	0	1	0	0	0	1	0
0	1	0	0	0	1	0	0
0	1	1	1	1	1	1	1
1	0	0	0	1	1	1	1
1	0	1	0	1	1	1	1
1	1	0	0	1	1	1	1
1	1	1	1	1	1	1	1

Dual of distributive law:

$$A . (B+C) = A.B + A.C$$

❖ Proof: Truth Table

A	B	C	B+C	A(B+C)	AB	AC	AB+AC
0	0	0	0	0	0	0	0
0	0	1	1	0	0	0	0
0	1	0	1	0	0	0	0
0	1	1	1	0	0	0	0
1	0	0	0	0	0	0	0
1	0	1	1	1	0	1	1
1	1	0	1	1	1	0	1
1	1	1	1	1	1	1	1

Basic Theorem and Properties of Boolean Algebra

❖ De Morgan's Theorem :

(i) The complement of the sum of 2 variables is equal to the product of the complements of individual variables:

$$\overline{A+B} = \overline{A} \cdot \overline{B}$$

(ii) The complement of the product of 2 variables is equal to the sum of the complements of individual variables:

$$\overline{A \cdot B} = \overline{A} + \overline{B}$$

A	B	$\overline{A}$	$\overline{B}$	A+B	A.B	$\overline{A+B}$	$\overline{A} \cdot \overline{B}$	$\overline{A \cdot B}$	$\overline{A} + \overline{B}$
0	0	1	1	0	0	1	1	1	1
0	1	1	0	1	0	0	0	1	1
1	0	0	1	1	0	0	0	1	1
1	1	0	0	1	1	0	0	0	0

Basic Theorem and Properties of Boolean Algebra

❖ Absorption Theorem:

(i) $A + AB = A$

$$\begin{aligned}\text{LHS: } &= A + AB \\ &= A.1 + AB \rightarrow \text{since } A.1 = A \\ &= A(1+B) \rightarrow \text{since } 1+B = 1 \\ &= A.1 \\ &= A = \text{RHS}\end{aligned}$$

(ii) $A(A+B) = A$

$$\begin{aligned}\text{LHS} &= A(A+B) \\ &= A.A + A.B \\ &= A + AB \rightarrow \text{since } A.A = A \\ &= A(1+B) \\ &= A.1 \\ &= A = \text{RHS}\end{aligned}$$

(iii) $A + \bar{A}B = A + B$

$$\begin{aligned}\text{LHS} &= A + \bar{A}B \\ &= (A + \bar{A})(A + B) \rightarrow \text{since } A + BC = (A+B)(A+C) \\ &= (1).(A + B) \rightarrow \text{since } A + \bar{A} = 1 \\ &= A + B = \text{RHS}\end{aligned}$$

(iv) $A.(\bar{A} + B) = AB$

$$\begin{aligned}\text{LHS} &= A.(\bar{A} + B) \\ &= A.\bar{A} + A.B \rightarrow (A.\bar{A} = 0) \\ &= AB = \text{RHS}\end{aligned}$$

❖ Redundancy Laws

Basic Theorem and Properties of Boolean Algebra

❖ Consensus Theorem:

$$AB + \bar{A}C + BC = AB + \bar{A}C$$

$$\text{LHS} = AB + \bar{A}C + BC$$

$$= AB + \bar{A}C + BC.1$$

$$= AB + \bar{A}C + BC(A + \bar{A}) \rightarrow \text{since } A + \bar{A} = 1$$

$$= AB + \bar{A}C + ABC + \bar{A}BC$$

$$= AB(1 + C) + \bar{A}C(1 + B)$$

$$1 + B = 1 + C = 1$$

$$= AB + \bar{A}C = \text{RHS}$$

❖ **BC is redundant term**

❖ Dual of consensus theorem:

$$(A+B)(\bar{A}+C)(B+C) = (A+B)(\bar{A}+C)$$

$$\text{LHS} = (A+B)(\bar{A}+C)(B+C)$$

$$\text{WKT } A+BC = (A+B)(A+C)$$

$$B+AC = (B+A)(B+C)$$

$$\text{LHS} = (B+AC)(\bar{A}+C)$$

$$= \bar{A}B + \bar{A}AC + BC + ACC$$

$$= \bar{A}B + BC + AC + A\bar{A}$$

$$= B(\bar{A} + C) + A(\bar{A} + C)$$

$$= (A+B)(\bar{A}+C) = \text{RHS}$$

Boolean Functions, Canonical and Standard Form

- ❖ Boolean function described by an algebraic expression consists of **binary variables, the constants 0 and 1, and the logic operation symbols**.
- ❖ Boolean function is **evaluated to logic-1 or logic-0** for a given value of the binary variables.
- ❖ Boolean function can be represented in a **truth table**.
- ❖ A Boolean function can be implemented as **digital circuit**, Which is constructed by using logic gates.
- ❖ Example: $F = X + Y'Z$
 $F = 1$ if $X = 1$ or if $Y = 0$ and $Z = 1$
 $F = 0$ Otherwise

Boolean Functions, Canonical and Standard Form

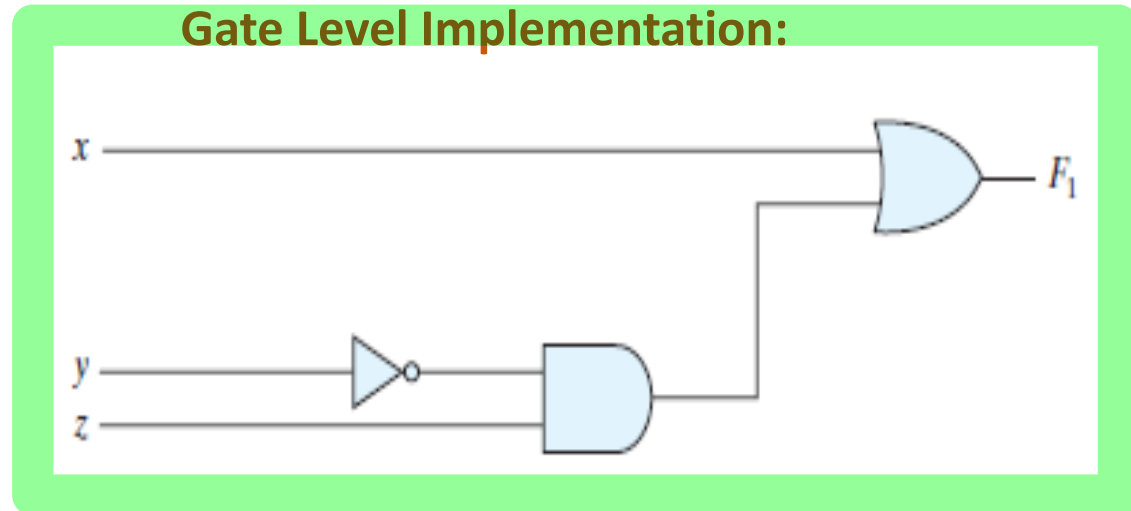
❖ Example: $F_1 = x + y'z$

❖ F_1 contains either 0 or 1 for each of these combinations. The table shows that the function is equal to 1 when $x = 1$ or when $yz = 01$ and is equal to 0 otherwise.

Truth Table:

x	y	z	F_1
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Gate Level Implementation:



❖ Simplify and realize the given Boolean function using basic gates

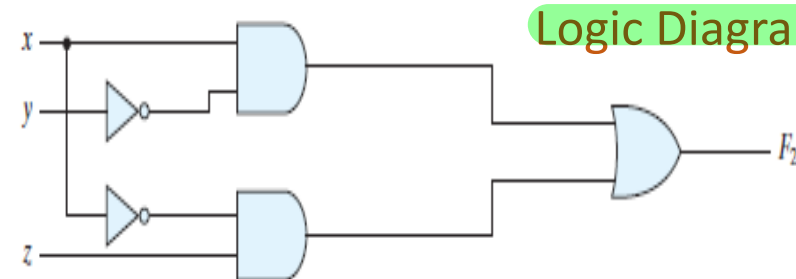
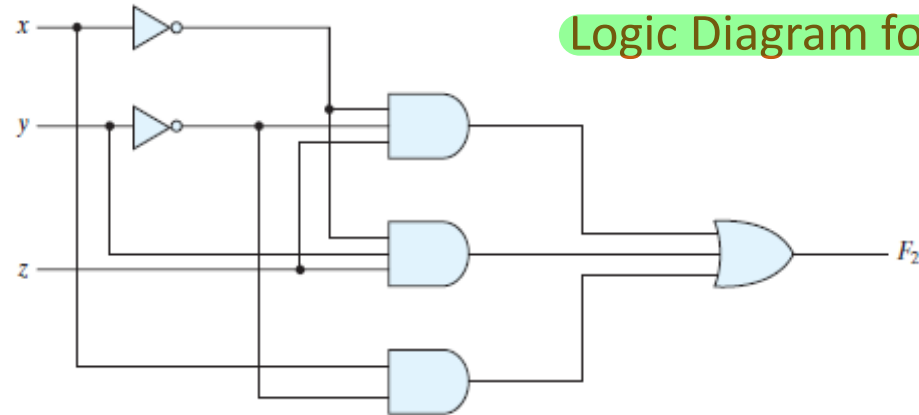
$$F_2 = x'y'z + x'yz + xy' \dots \dots \dots (1)$$

$$F_2 = x'z (y' + y) + xy'$$

$$F_2 = x'z + xy' \dots \dots \dots (2)$$

Truth Table

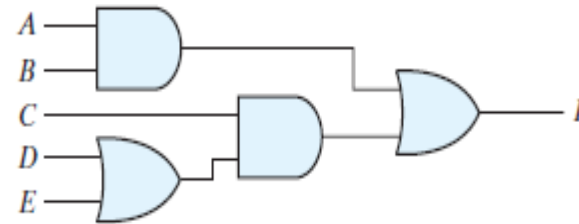
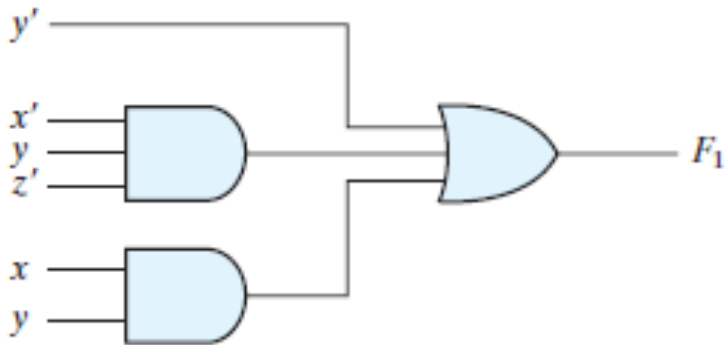
x	y	z	F ₂
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0



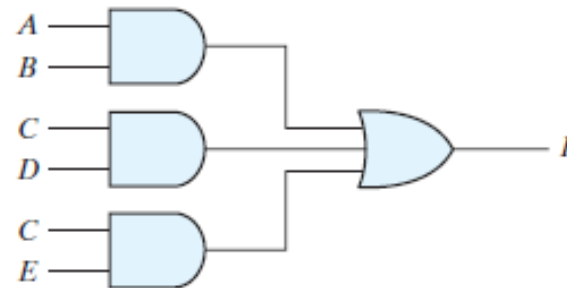
❖ Sum of Products: (SOP)

- The sum of products is a Boolean expression containing AND terms, called **product terms**, with one or more literals each. The **sum** denotes the ORing of these terms.
- Two- Level Logic implementation of SOP

❖ Example 1: $F_1 = y' + xy + x'yz'$



Example 2:
 $F = AB + C(D+E)$



$F = AB + CD + CE$

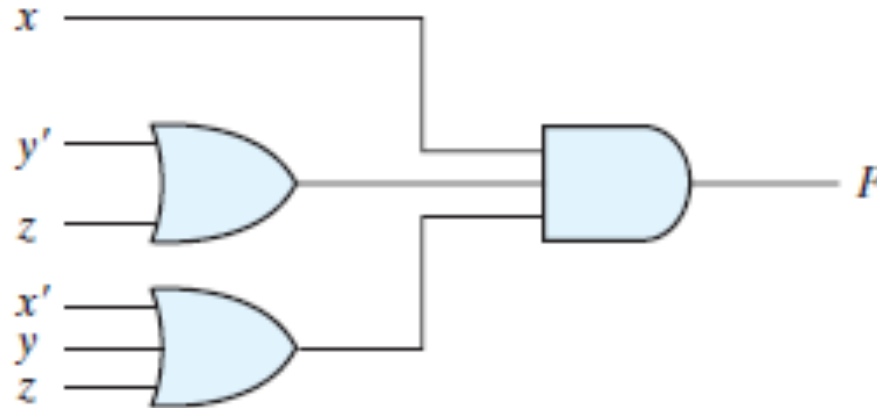
Boolean Functions, Canonical and Standard Form

❖ Product Of Sum: POS

➤ A product of sum (POS) is a Boolean expression containing OR terms, called sum terms. Each term may have any number of literals. The product denotes the ANDing of these terms

➤ Example 1 :

$$F = x.(y' + z).(x' + y + z)$$



➤ Example 2: $Y = (A+B). (C+D)$

❖ Canonical SOP Form:

➤ Each product term contains all the literals of that function either in true or complement form

➤ **Example:** $F(x,y,z) = xyz + x'y'z + x'yz' + x'y'z'$

➤ Each product term is called as **minterm**

➤ For three variable function: Truth Table

<i>x</i>	<i>y</i>	<i>z</i>	Minterms	
			Term	Designation
0	0	0	$x'y'z'$	m_0
0	0	1	$x'y'z$	m_1
0	1	0	$x'yz'$	m_2
0	1	1	$x'yz$	m_3
1	0	0	$xy'z'$	m_4
1	0	1	$xy'z$	m_5
1	1	0	xyz'	m_6
1	1	1	xyz	m_7

❖ Consider the function: f_1 and f_2 in the truth table

x	y	z	Function f_1	Function f_2
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

❖ Canonical SOP Form for f_1

$$f_1 = x'y'z + xy'z' + xyz = m_1 + m_4 + m_7$$

❖ Canonical SOP Form for f_2

$$f_2 = x'yz + xy'z + xyz' + xyz = m_3 + m_5 + m_6 + m_7$$

Boolean Functions, Canonical and Standard Form

❖ Canonical POS Form:

➤ Each sum term contains all the literals of that function either in true or complement form

➤ **Example:** $F(x, y, z) = (x + y + z) (x' + y' + z) (x' + y + z')$

➤ Each sum term is called as **Maxterm**

➤ For three variable function: Truth Table

<i>x</i>	<i>y</i>	<i>z</i>	Maxterms	
			Term	Designation
0	0	0	$x + y + z$	M_0
0	0	1	$x + y + z'$	M_1
0	1	0	$x + y' + z$	M_2
0	1	1	$x + y' + z'$	M_3
1	0	0	$x' + y + z$	M_4
1	0	1	$x' + y + z'$	M_5
1	1	0	$x' + y' + z$	M_6
1	1	1	$x' + y' + z'$	M_7

❖ Consider the function: f_1 and f_2 in the truth table

x	y	z	Function f_1	Function f_2
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

❖ Canonical POS Form for f_1

$$\begin{aligned} f_1 &= (x + y + z)(x + y' + z)(x + y' + z')(x' + y + z') \\ &\quad (x' + y' + z) \\ &= M_0 \cdot M_2 \cdot M_3 \cdot M_5 \cdot M_6 \end{aligned}$$

❖ Canonical POS Form for f_2

$$\begin{aligned} f_2 &= (x + y + z)(x + y + z')(x + y' + z)(x' + y + z) \\ &= M_0 \cdot M_1 \cdot M_2 \cdot M_4 \end{aligned}$$

❖ Find the Minterms for the given expression: (Convert into canonical SOP Form)

$$F = A + BC$$

$$F = A.1.1 + BC.1$$

$$F = A. (B+B'). (C+C') + BC.(A+A')$$

$$F = A. (BC + BC' + B'C + B'C') + ABC + A'BC$$

$$F = ABC + ABC' + AB'C + AB'C' + ABC + A'BC$$

$$F = ABC + ABC' + AB'C + AB'C' + A'BC$$

$$F = m_7 + m_6 + m_5 + m_4 + m_3$$

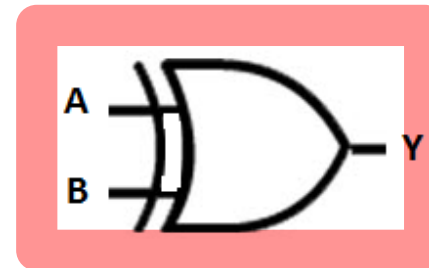
$$F = \Sigma (3,4,5,6,7)$$

Other Logical Operation, Digital Logic gates (XOR and XNOR)

- ❖ **XOR Gate** is a digital logic Gate which has two or more inputs and only one output that performs Exclusive OR operation. Hence it is also called as Ex-OR or XOR
- ❖ For two input XOR gate output is **logic-1** only when one of its input is logic-1 (unequal input i.e..A = 0 and B = 1 or A = 1 and B = 0).
- ❖ Output of XOR gate is **logic-0** if both inputs are same (i.e., A = 0 and B = 0 or A = 1 and B = 1)
- ❖ Truth Table of XOR Gate:

Input A	Input B	Output Y
0	0	0
0	1	1
1	0	1
1	1	0

❖ Symbol



$$A.B' + A'.B = A \oplus B$$

Other Logical Operation, Digital Logic gates (XOR and XNOR)

❖ XOR gate performs modulo sum operation without including carry.

i.e., $0+0=0$, $0+1=1$, $1+0=1$, $1+1=0$ (carry 1)

Proof is just Truth Table

❖ Truth Table

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

$$Y = A'.B + A.B'$$

$$Y = A \text{ XOR } B$$

$$Y = A \oplus B$$

$$A.B' + A'.B = A \oplus B$$

$$\text{LHS} = A.B' + A'.B$$

Case1: If $A = 0$ and $B = 0$

$$0.0' + 0'.0 = 0.1 + 1.0 = 0$$

Case2: If $A = 0$ and $B = 1$

$$0.1' + 0'.1 = 0.0 + 1.1 = 1$$

Case3: If $A = 1$ and $B = 0$

$$1.0' + 1'.0 = 1.1 + 0.0 = 1$$

Case4: If $A = 1$ and $B = 1$

$$1.1' + 1'.1 = 1.0 + 0.1 = 0$$

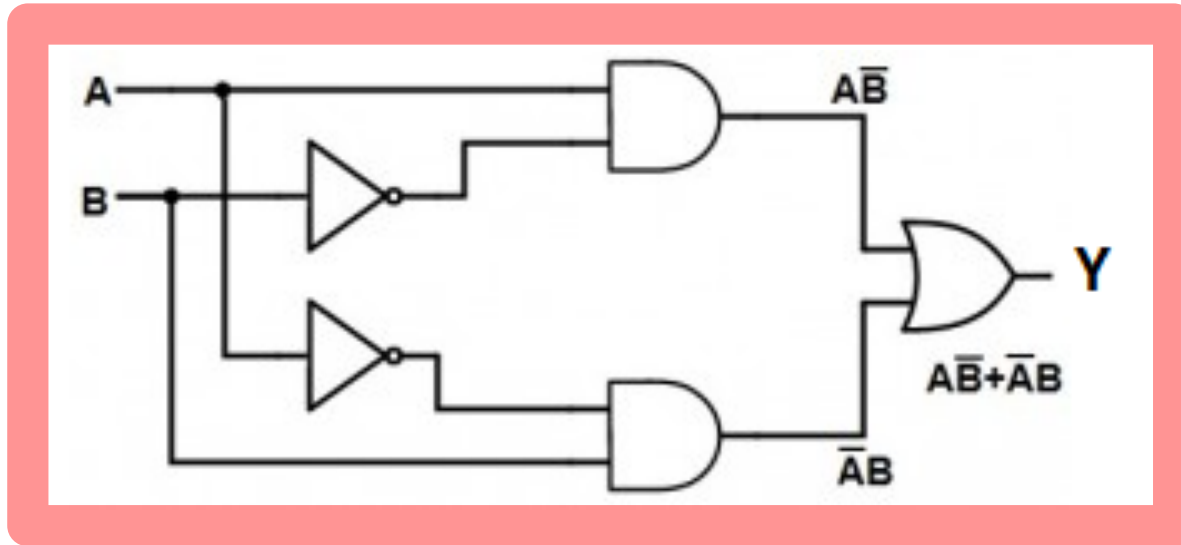
$$\text{Hence } Y = A \oplus B = A.B' + A'.B$$

Other Logical Operation, Digital Logic gates (XOR and XNOR)

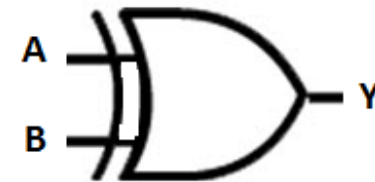
- ❖ Implementation of XOR Gate using basic gates:

$$Y = A \oplus B = A.B' + A'.B$$

- ❖ Logic Diagram



- ❖ Symbol



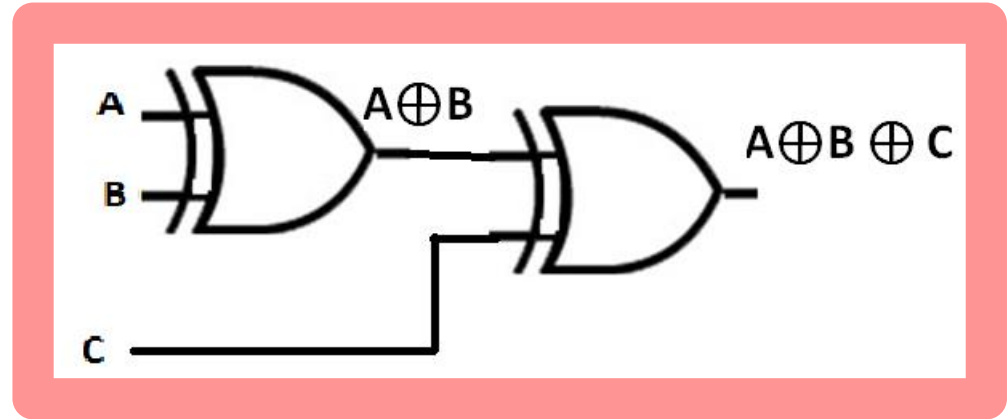
Other Logical Operation, Digital Logic gates (XOR and XNOR)

❖ **3 - Input XOR Gate:** $Y = A \oplus B \oplus C$

Truth Table

A	B	C	$A \oplus B$	$A \oplus B \oplus C$
0	0	0	0	0
0	0	1	0	1
0	1	0	1	1
0	1	1	1	0
1	0	0	1	1
1	0	1	1	0
1	1	0	0	0
1	1	1	0	1

Symbol



Associative Law:

$$(A \oplus B) \oplus C = A \oplus (B \oplus C)$$

❖ **Even or odd Parity Bits**

Other Logical Operation, Digital Logic gates (XOR and XNOR)

❖ **XNOR Gate:** Logical Complement of XOR Gate

❖ For two input XNOR Gate if both inputs are same i.e., $A = 0$ and $B = 0$ or $A = 1$ and $B = 1$. Then output of logic gate is logic-1 (High). Output of XNOR is logic-0 if inputs are unequal.

❖ Equality detector

Truth Table

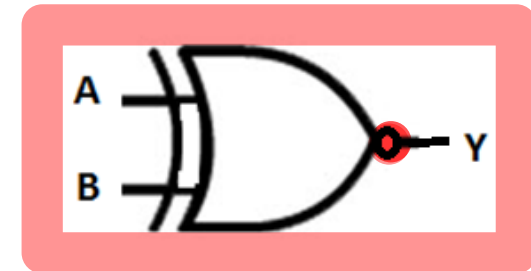
Input A	Input B	Output Y
0	0	1
0	1	0
1	0	0
1	1	1

❖ Boolean Expression

$$Y = A.B + A'.B'$$

$$Y = A \odot B$$

❖ Symbol



Other Logical Operation, Digital Logic gates (XOR and XNOR)

❖ XNOR Gate:

$$Y = (A \oplus B)'$$

$$Y = (A.B' + A'.B)'$$

De Morgan's Theorem

$$Y = (A.B')' . (A'.B)'$$

$$Y = (A' + B) . (A + B')$$

$$Y = A'.A + A'.B' + A.B + B.B'$$

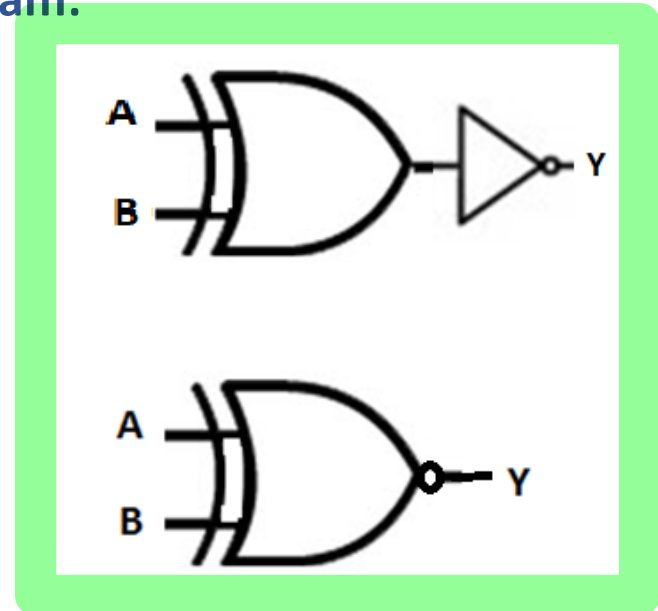
$$Y = 0 + A'.B' + A.B + 0$$

$$Y = A'.B' + A.B$$

$$Y = (A \oplus B)' = A \odot B$$

$$Y = (A \text{ XOR } B)' = (A \text{ XNOR } B)$$

❖ Logic Diagram:



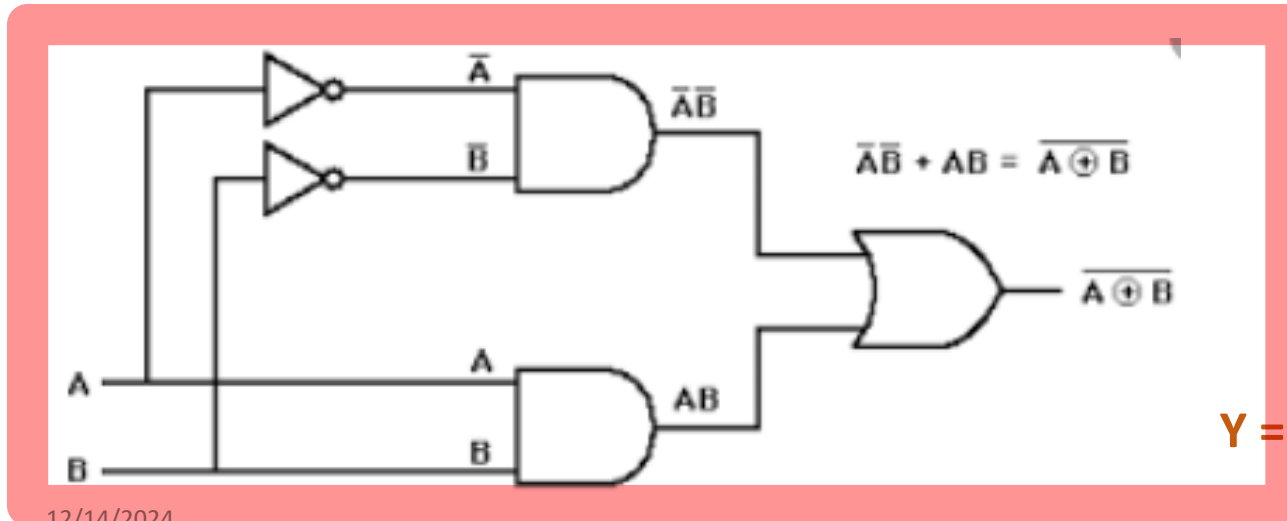
❖ Symbol:

Other Logical Operation, Digital Logic gates (XOR and XNOR)

❖ Implement the XNOR Gate using basic gates:

$$Y = A'.B' + A.B$$

❖ Logic Diagram:



$$Y = (A \oplus B)' = A \odot B$$

Other Logical Operation, Digital Logic gates (XOR and XNOR)

❖ Properties of XOR Gate:

➤ Identity element: $A \oplus 0 = A$

➤ $A \oplus 1 = A'$

➤ $A \oplus A = 0$

➤ Commutative Law: $A \oplus B = B \oplus A$

➤ Associative Law : $A \oplus (B \oplus C) = (A \oplus B) \oplus C$



Input A	Input B	Output Y
0	0	0
0	1	1
1	0	1
1	1	0

Other Logical Operation, Digital Logic gates (XOR and XNOR)

❖ Applications of XOR and XNOR

- XOR gate is used in processor's Arithmetic Logic Unit (ALU) for binary addition.
- XOR logic gate is used to generate pseudorandom numbers in hardware.
- To generate parity bits and error detection
- Equality detector

Realization of Boolean expression using Universal Gates

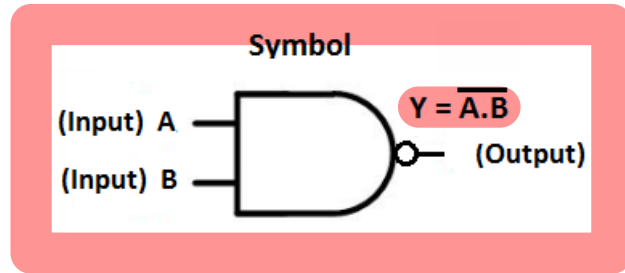
- ❖ **Universal Gates** :
 - (i) NAND Gate
 - (ii) NOR Gate

- ❖ Any digital logic circuit can be implemented by using NAND or NOR logic gates.

- ❖ NAND and NOR gates are easier to fabricate with electronic components and are used in all Integrated Circuit (IC's) digital logic families.

Realization of Boolean expression using Universal Gates

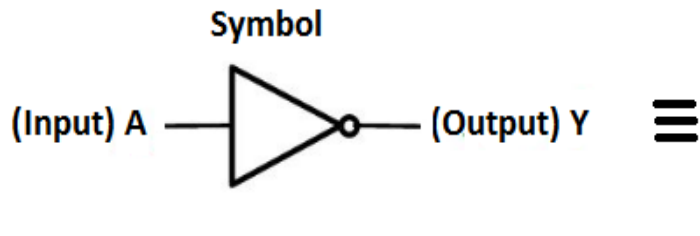
❖ Realization of logic gates using NAND Gates:



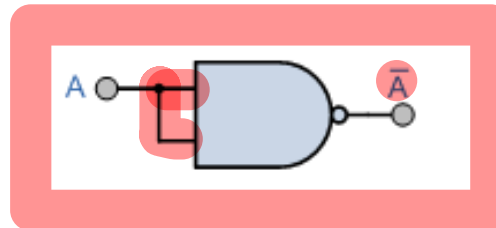
Truth Table

NAND		
Input		Output
A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

❖ NOT Gate:



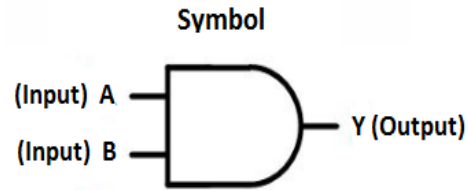
≡



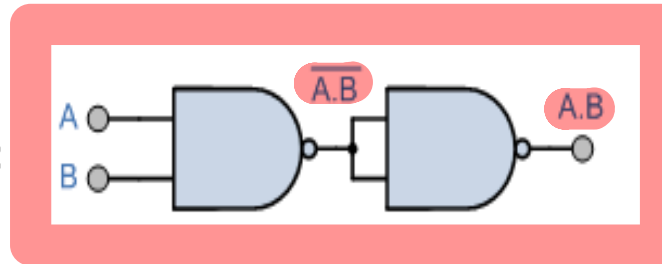
$$Y = (A.A)' = (A)' = A'$$

Realization of Boolean expression using Universal Gates

❖ AND Gate:

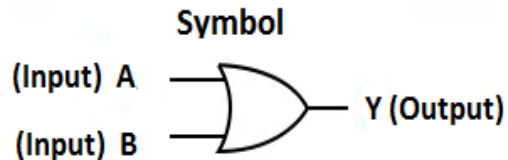


≡

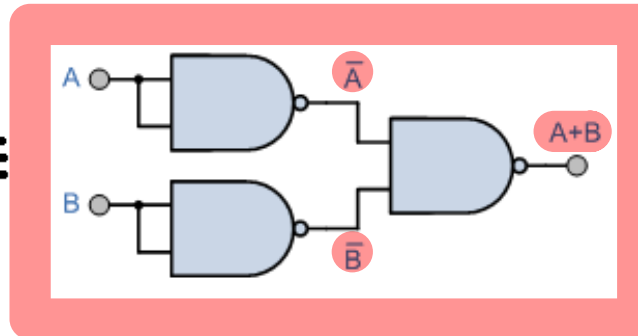


$$Y = ((A.B)')' = A.B$$

❖ OR Gate:



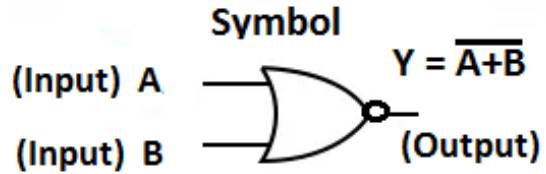
≡



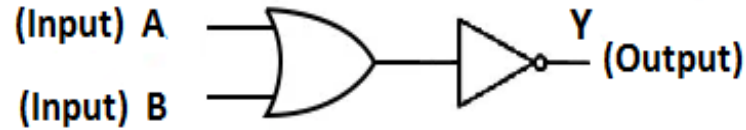
$$Y = ((A+B)')' = (A' . B')' = A+B$$

Realization of Boolean expression using Universal Gates

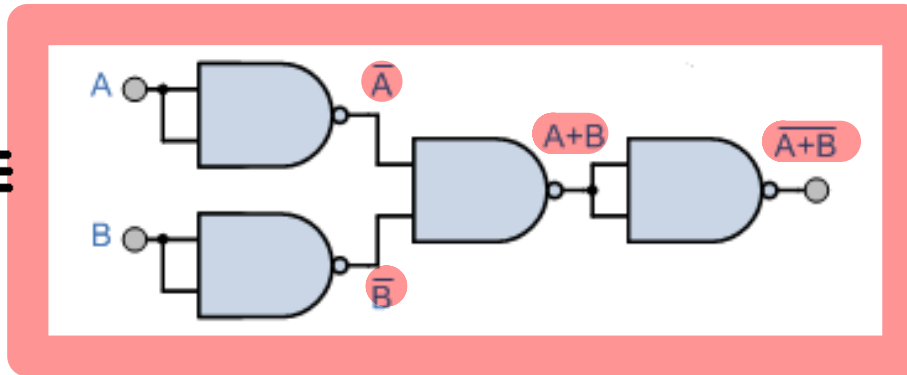
❖ **NOR** Gate:



≡



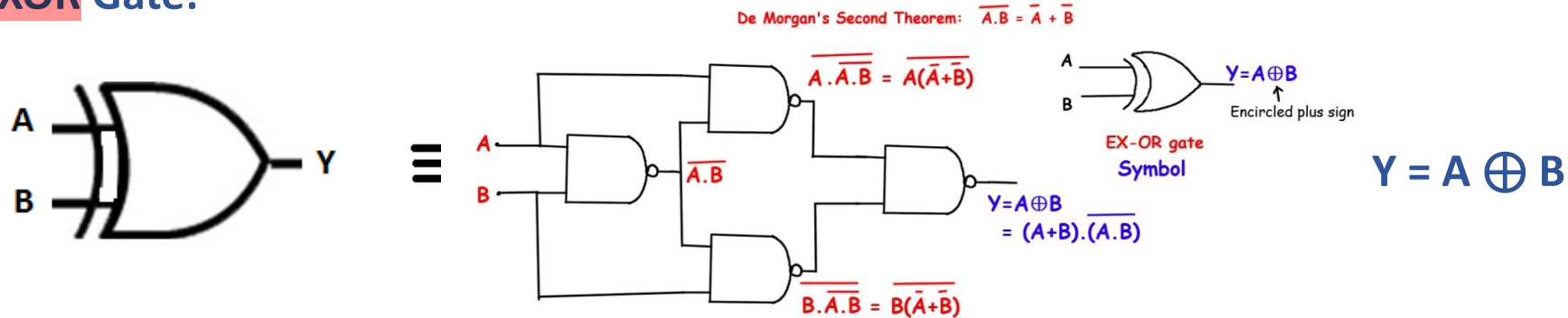
≡



Realization of Boolean expression using Universal Gates

EX-OR Gate using NAND Gate

❖ **XOR Gate:**



EX-OR Gate using NAND Gate

$$Y = A.B' + A'B = A (A.B)' + B. (A.B)'$$

$$Y = ((A (A.B)' + B. (A.B)'))'$$

$$Y = (((A. (A.B)')' . (B.(A.B)')')))'$$

$$Y = A (A.B)' + B. (A.B)'$$

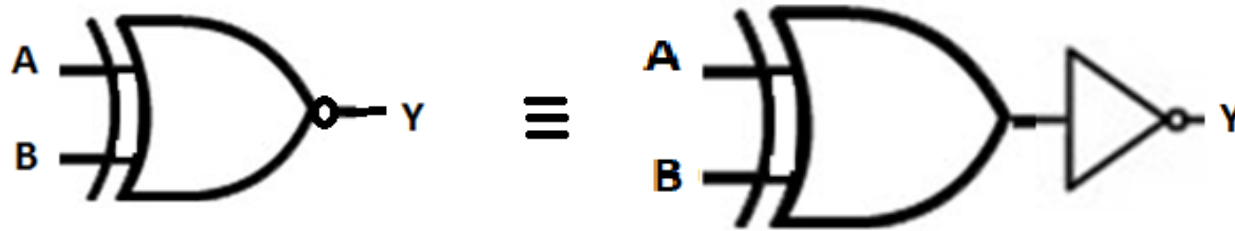
$$Y = A (A' + B') + B. (A' + B')$$

$$Y = A.A' + A.B' + A'.B + B.B'$$

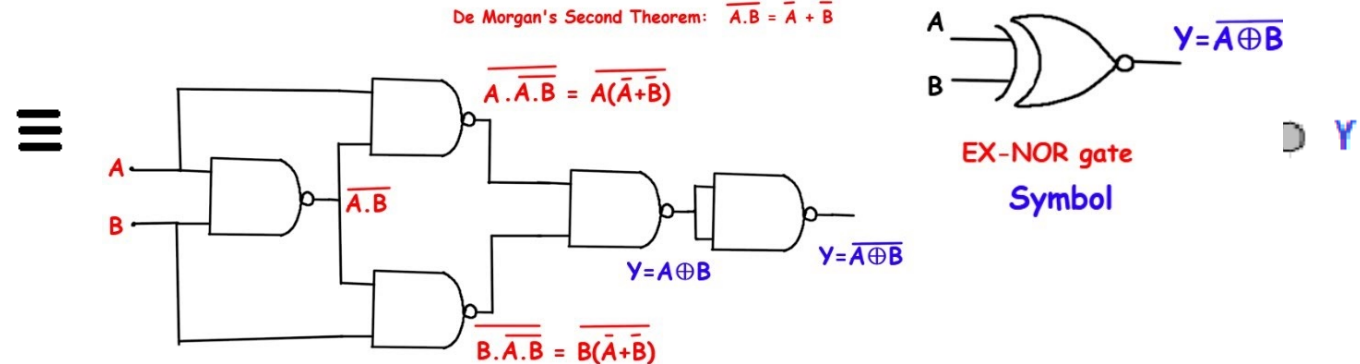
$$Y = A.B' + A'.B$$

Realization of Boolean expression using Universal Gates

❖ XNOR Gate:

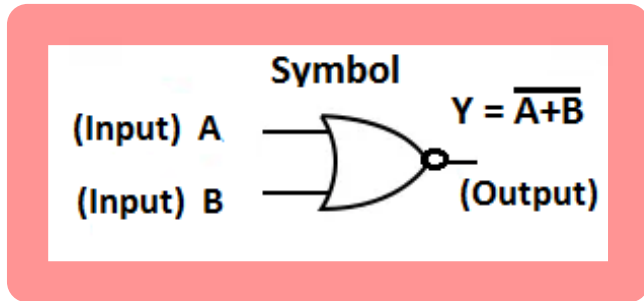


EX-NOR Gate using NAND Gate



Realization of Boolean expression using Universal Gates

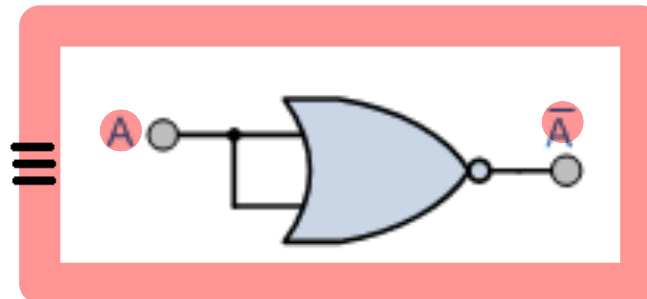
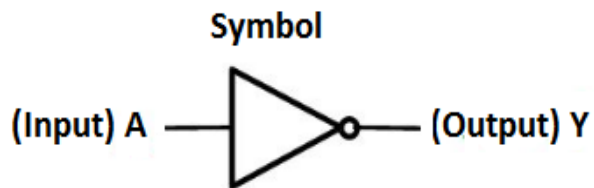
❖ Realization of logic gates using NOR Gates:



Truth Table

NOR		
Input		Output
A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

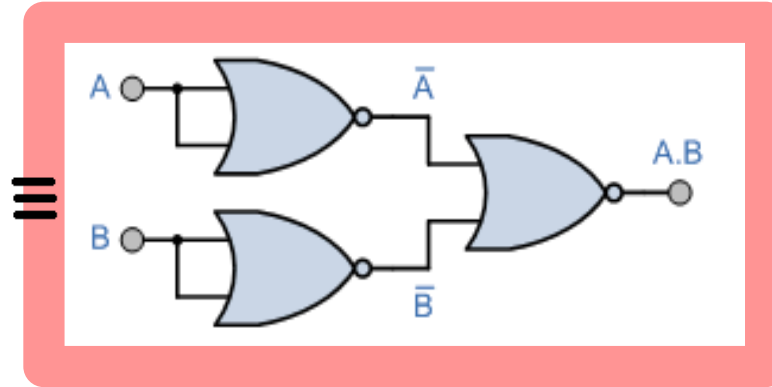
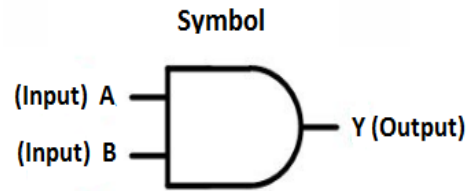
❖ NOT Gate:



$$Y = (A + A)' = (A)' = A'$$

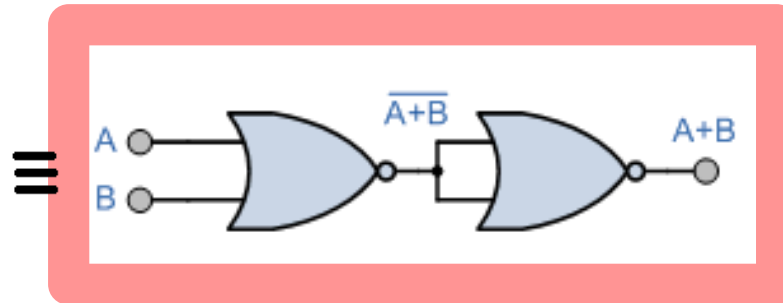
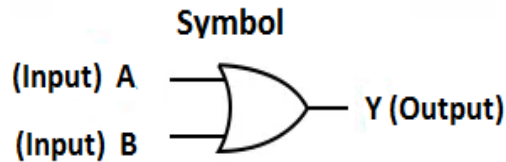
Realization of Boolean expression using Universal Gates

❖ AND Gate:



$$Y = ((A.B)')' = (A' + B')' = A.B$$

❖ OR Gate:



$$Y = ((A+B)')' = A + B$$

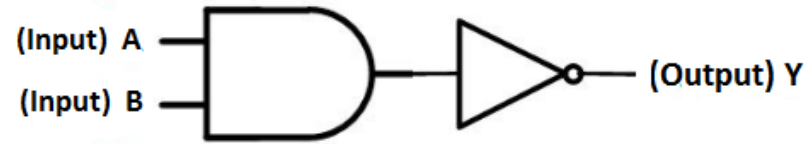
Realization of Boolean expression using Universal Gates

❖ **NAND** Gate:

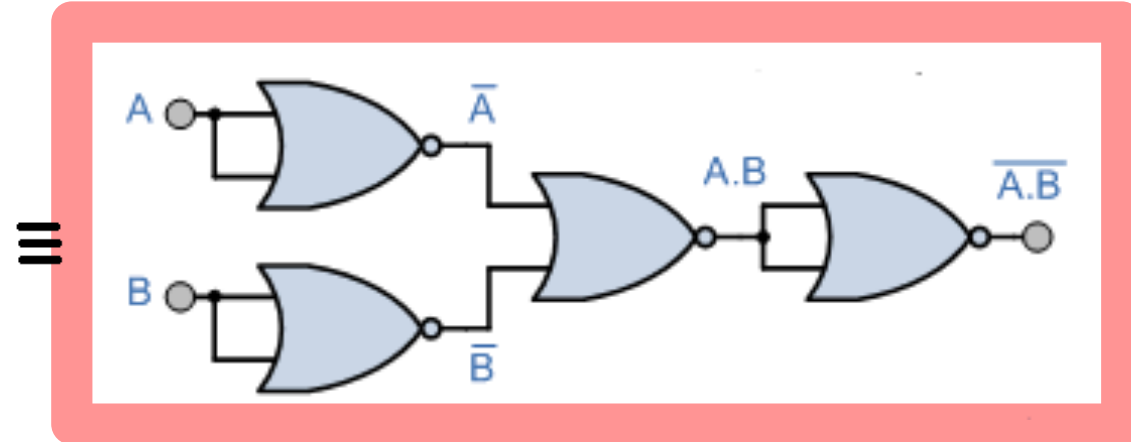
Symbol



≡

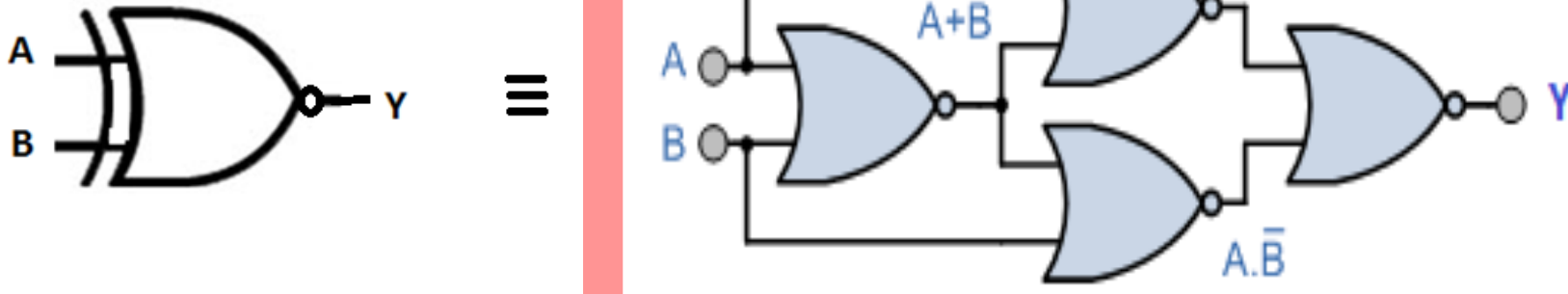


≡



Realization of Boolean expression using Universal Gates

❖ **XNOR** Gate:



$$Y1 = ((A+B)' + A)' = (A+B). A' = A'.B$$

$$Y2 = ((A+B)' + B)' = (A+B). B' = A.B'$$

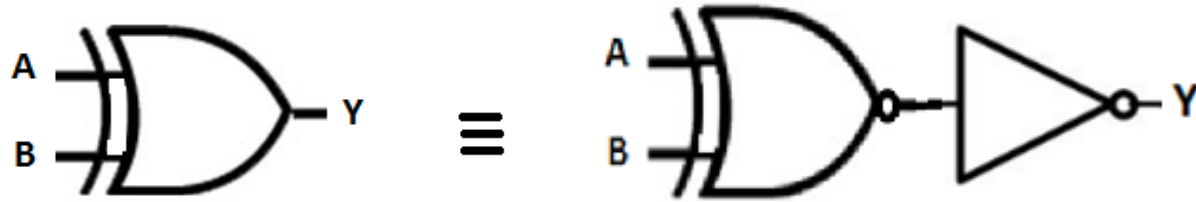
$$Y = (A'.B + A.B')' = (A'.B)' . (A.B')' = (A + B') . (A' + B)$$

$$Y = A.B + A'.B'$$

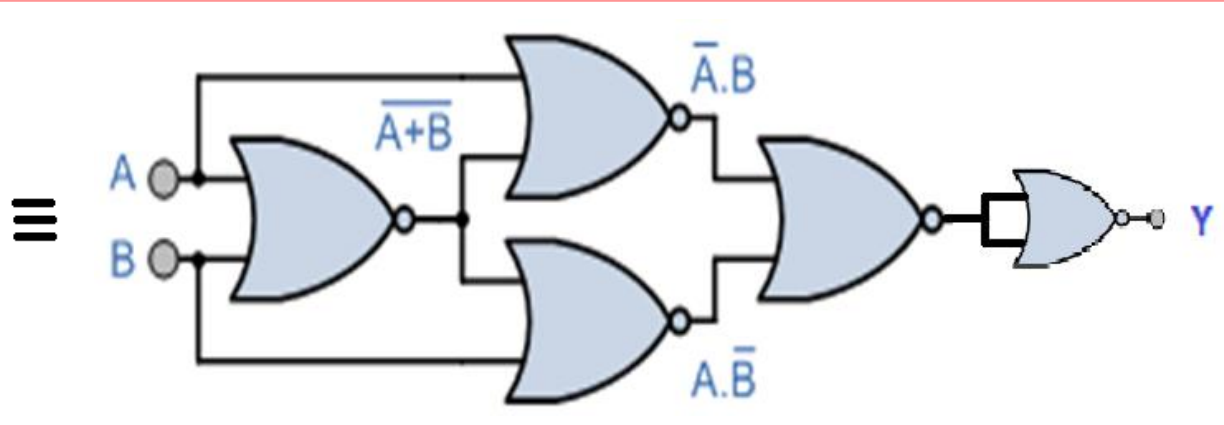
$$Y = A \odot B$$

Realization of Boolean expression using Universal Gates

❖ **XOR** Gate:



$$Y = (A \odot B)' = A \oplus B$$



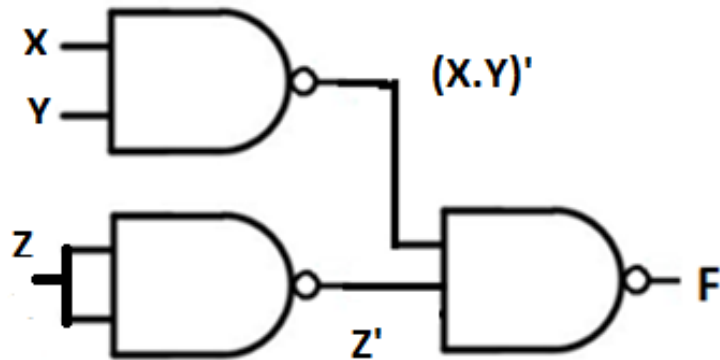
Realization of Boolean expression using Universal Gates

❖ Implement the given function using NAND Gates only.

$$F = X.Y + Z$$

$$F = ((X.Y + Z)')'$$

$$F = ((X.Y)' . Z')'$$



Problems on Boolean algebra

❖(1) Simplify the given Boolean expression and implement using basic gates

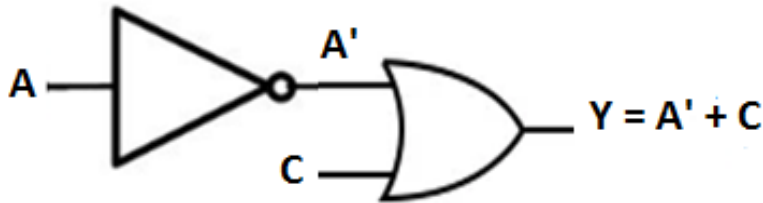
$$F = A.B.C + A' + A. B'.C$$

$$F = A.C (B + B') + A' \quad \text{where } B + B' = 1$$

$$F = A.C + A' \quad \text{Distributive Law}$$

$$F = (A' + A) . (A' + C) \quad A + B.C = (A+B) . (A+C)$$

$$F = A' + C$$



Logic Diagram

Problems on Boolean algebra

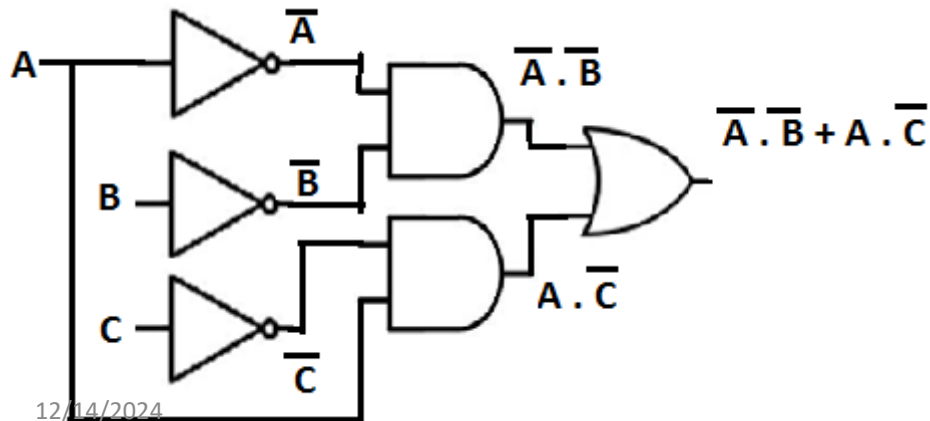
❖(2) Simplify and realize the given function using basic gates:

$$Y = A'B'C' + AB'C' + A'B' + AC'$$

$$Y = B'C' (A' + A) + A'B' + AC'$$

$$Y = B'C' \cdot (1) + A'B' + AC'$$

By Consensus Theorem: $Y = A'B' + AC'$ $AB + \bar{A}C + BC = AB + \bar{A}C$



Problems on Boolean algebra

❖ (3) Simplify and Implement the given function using NAND Gates only.

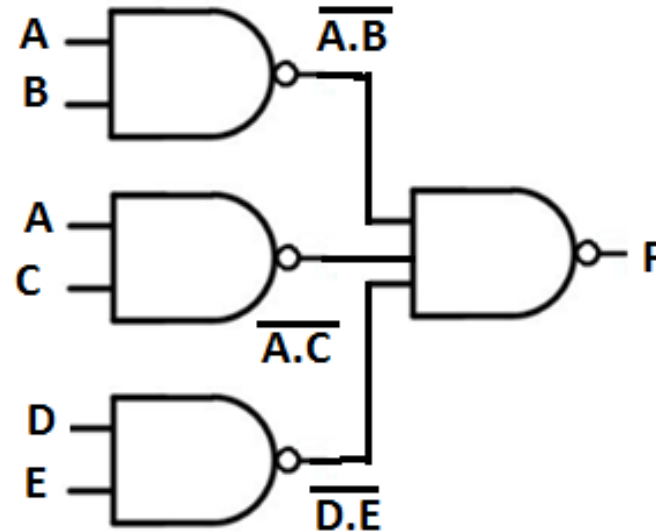
$$F = A(B + C) + DE$$

❖ SOP Form: $F = AB + AC + DE$

$$F = \overline{\overline{AB + AC + DE}}$$

$$F = \overline{(\overline{A.B}) . (\overline{A.C}) . (\overline{D.E})}$$

DeMorgan's Law



Logic Diagram

Problems on Boolean algebra

❖ (4) Simplify the given Boolean Expression:

$$Y = A + A'.B + A'.B'.C + A'.B'.C'.D + A'.B'.C'.D'.E$$

$$Y = A + A' (B + B'.C + B'.C'.D + B'.C'.D'.E)$$

Absorption Law: $A + A'.B = A + B$

$$Y = A + B + B'.C + B'.C'.D + B'.C'.D'.E$$

$$Y = A + B + B' (C + C'.D + C'.D'.E)$$

$$Y = A + B + C + C'.D + C'.D'.E$$

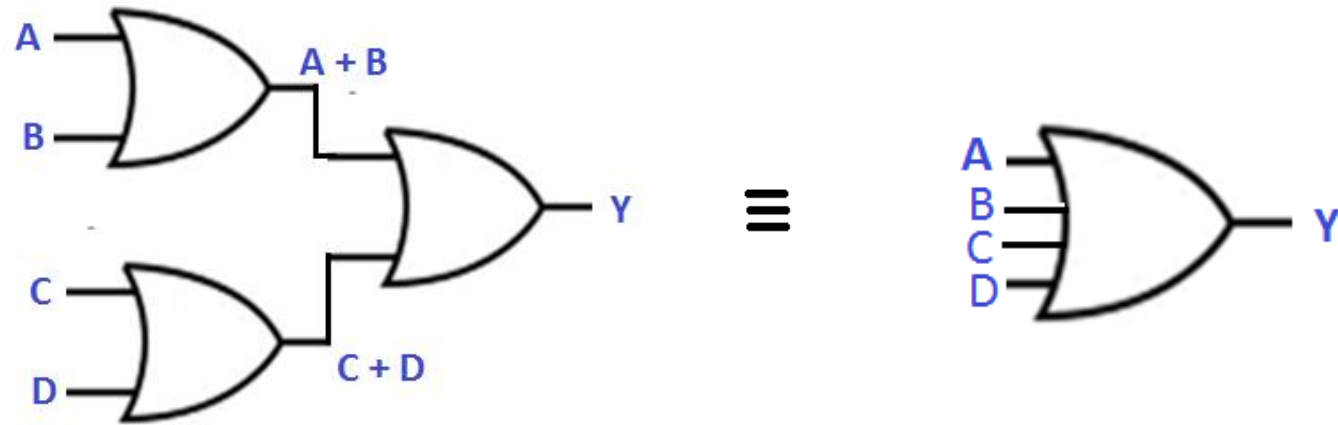
$$Y = A + B + C + C' (D + D'.E)$$

$$Y = A + B + C + D + D'.E$$

$$\mathbf{Y = A + B + C + D + E}$$

Problems on Boolean algebra

❖ Logic Diagram



❖ Evaluate the Boolean function $Y = C + C'B + BA'$

when $A = 1$, $B = 0$ and $C = 1$

$$Y = 1 + 1'.0 + 0.1' = 1 + 0 + 0$$

$$Y = 1$$

Problems on Boolean algebra

- ❖ (5) Simplify the given Boolean expression and implement using Basic and NAND gates

$$F = AB + A(B+C) + B(B+C)$$

$$F = AB + AB + AC + BB + BC \quad \text{Distributive Law}$$

$$F = AB + AC + B + BC$$

$$F = AB + AC + B(1 + C)$$

$$F = AB + AC + B$$

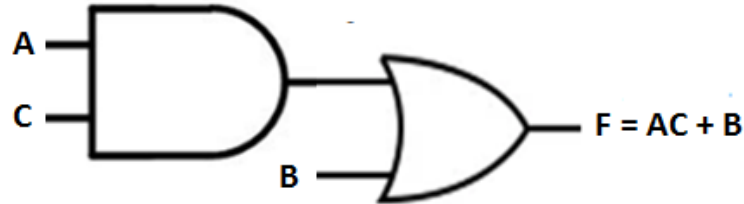
$$F = AC + B(A + 1)$$

$$F = AC + B$$

Problems on Boolean algebra

❖ $F = A.C + B$

❖ Using Basic Gates:

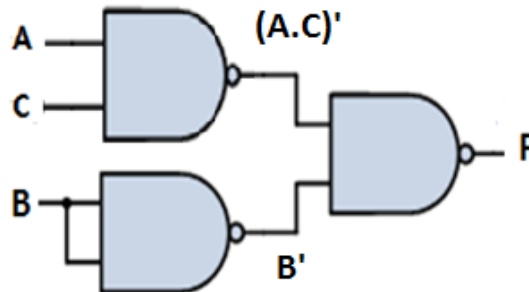


❖ Using NAND Gates:

$$F = AC + B$$

$$F = ((AC + B)')'$$

$$F = ((AC)' \cdot B')'$$



Problems on Boolean algebra

- ❖(6) Simplify and realize the given function using: (i) Basic Gates:
(ii) NAND Gates

$$Y = (A'.B + A' + A.B)'$$

$$Y = (A'.B)' . (A')' . (A.B)'$$

$$Y = ((A')' + B') . (A) . (A' + B')$$

$$Y = (A + B') . ((A.A') + A.B')$$

$$Y = (A + B') . (A.B')$$

$$Y = A.A.B' + A.B'.B'$$

$$Y = A.B' + A.B'$$

$$Y = A.B'$$

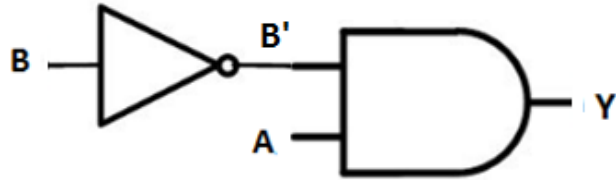
De Morgan's Theorem

$$\begin{aligned}(A'B + A' + A.B)' &= (A' + A.B)' = (A' + B)' \\ &= (A')'.B' = A.B'\end{aligned}$$

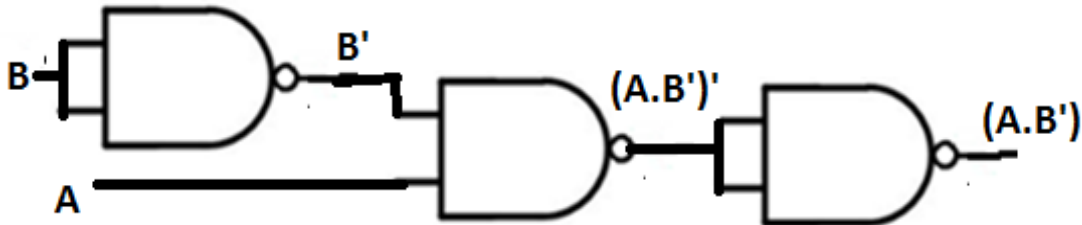
Problems on Boolean algebra

❖ (7) $Y = A.B'$

❖ Using Basic Gates:



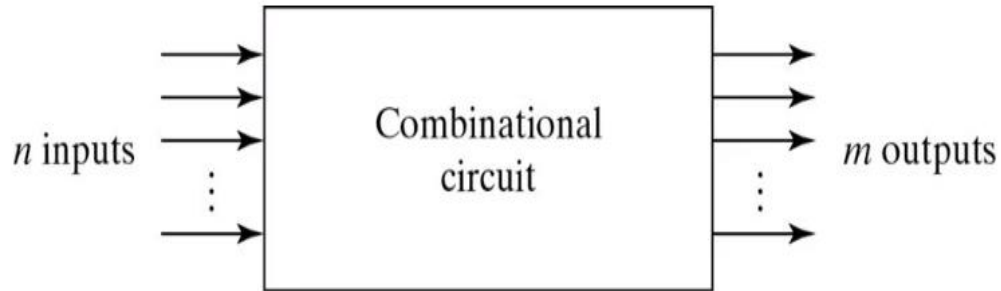
❖ Using NAND Gates:



[https://
www.youtube.com/
watch?
v=5XbRIVWFRIw](https://www.youtube.com/watch?v=5XbRIVWFRIw)

Combinational Logic Circuits: Half Adder and Full adder

- ❖ **Combinational circuits** are constructed by interconnection of logic gates.
whose outputs at any time are determined from only the present combination of inputs



- ❖ A combinational circuit performs an operation that can be specified logically by a set of **Boolean functions**
- ❖ Examples: Binary Adders, Multiplexers, etc.

Combinational Logic Circuits: Half Adder and Full adder

❖ Half Adder

- x and y are the two binary inputs
- Sum (s) and Carry (c) are the two binary outputs

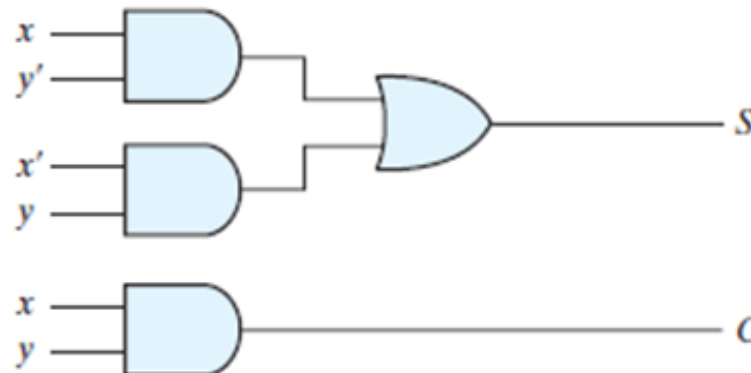
Boolean Expression:

$$s = x'.y + x.y'$$

$$c = x.y$$

Truth Table

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



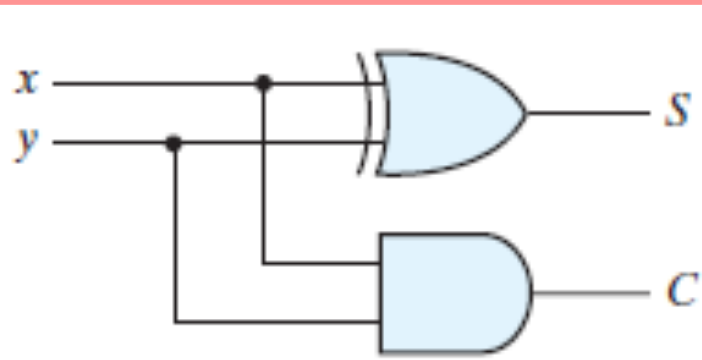
Logic Diagram

Combinational Logic Circuits: Half Adder and Full adder

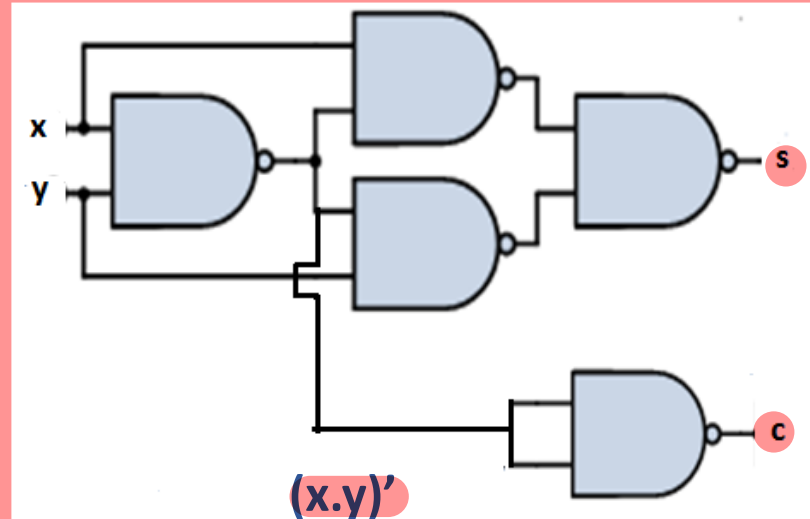
❖ **Half Adder:** $s = x \oplus y$
 $c = x.y$

Sum expression: $s = x'.y + x.y'$

❖ **Half Adder using Logic Gates**



❖ **Half Adder using NAND Gates only**



Combinational Logic Circuits: Half Adder and Full adder

❖ Full Adder:

- x, y and z are three binary inputs.
- S is sum and C is carry outputs

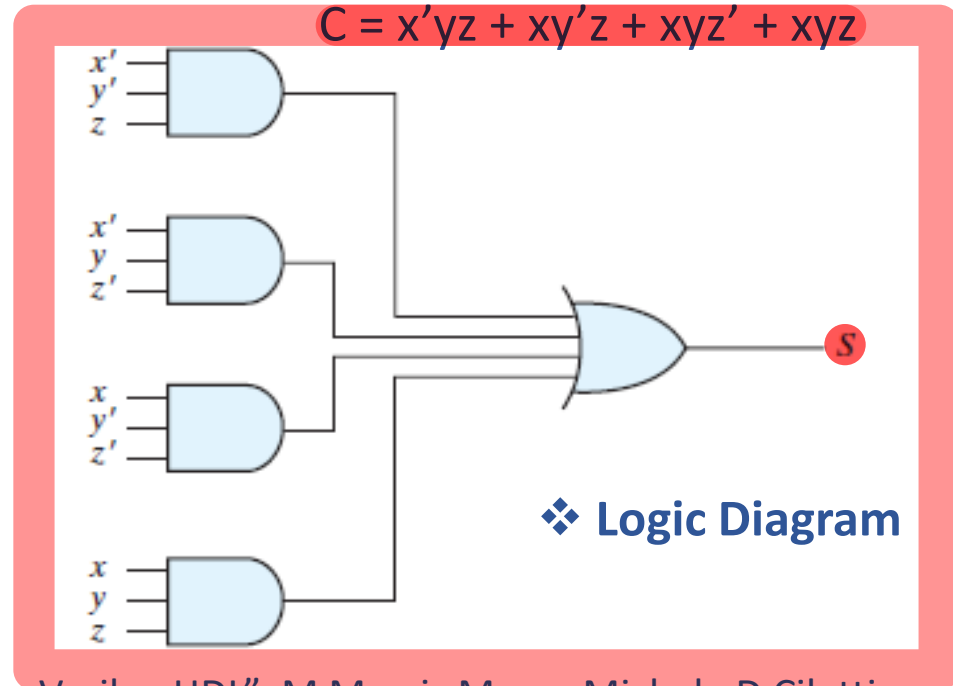
❖ Truth Table:

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

❖ Boolean Expression:

$$S = x'y'z + x'yz' + xy'z' + xyz$$

$$C = x'yz + xy'z + xyz' + xyz$$



❖ Logic Diagram

Combinational Logic Circuits: Half Adder and Full adder

❖ Full Adder:

➤ Carry Expression:

$$C = x'yz + xy'z + xyz' + xyz$$

$$C = x'yz + xy'z + xy(z' + z)$$

$$C = x'yz + xy'z + xy$$

$$C = x'yz + x(y'z + y) \quad \text{Absorption Law}$$

$$C = x'yz + x(z + y)$$

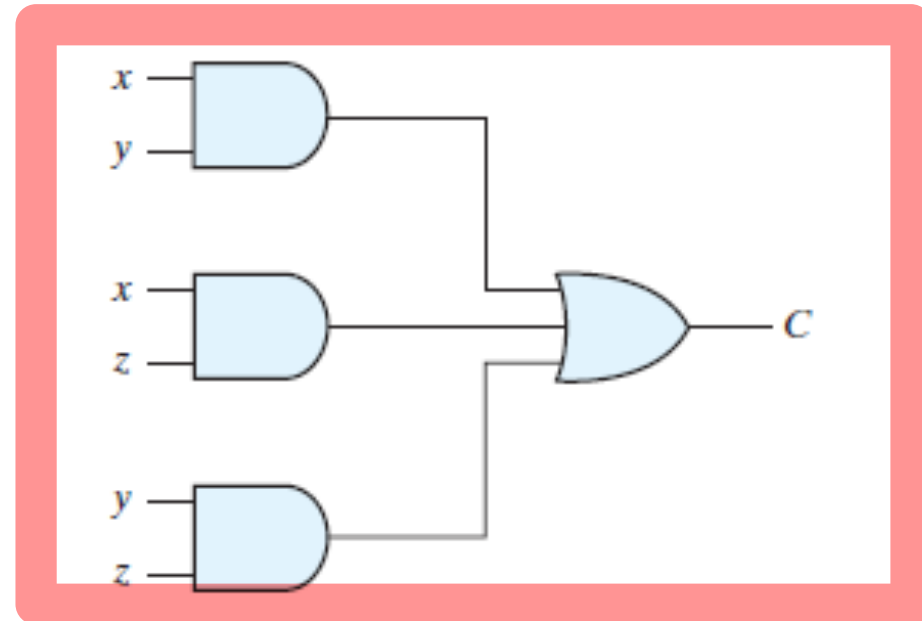
$$C = x'yz + xz + xy$$

$$C = z(x'y + x) + xy$$

$$C = z(y + x) + xy$$

$$C = yz + xz + xy$$

❖ Logic Diagram



Implement Full adder using two Half Adder

❖ Boolean Expression for Sum:

$$S = x'y'z + x'yz' + xy'z' + xyz$$

$$S = x' (y'z + yz') + x (y'z' + yz)$$

$$S = x' (y \oplus z) + x ((y \oplus z)')$$

$$S = x \oplus (y \oplus z)$$

❖ Boolean Expression for Carry:

$$C = x'yz + xy'z + xyz' + xyz$$

$$C = z. (x'y + xy') + xy (z' + z)$$

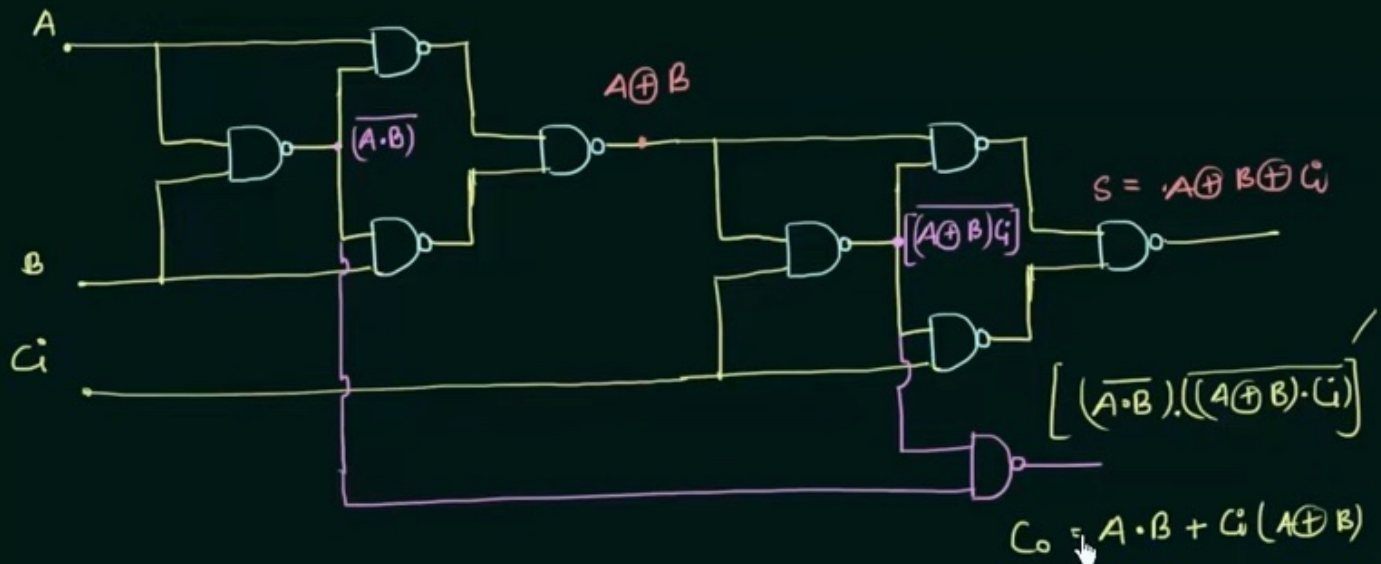
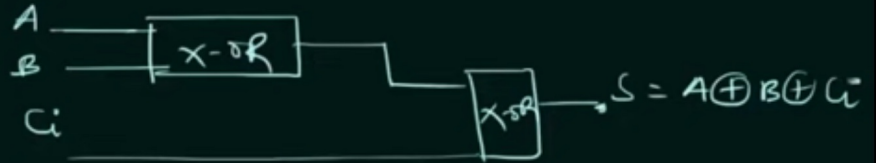
$$C = z. (x \oplus y) + xy .(1)$$

$$C = (x \oplus y).z + xy$$

Full Adder using NAND Gate

$$S = A \oplus B \oplus C_i$$

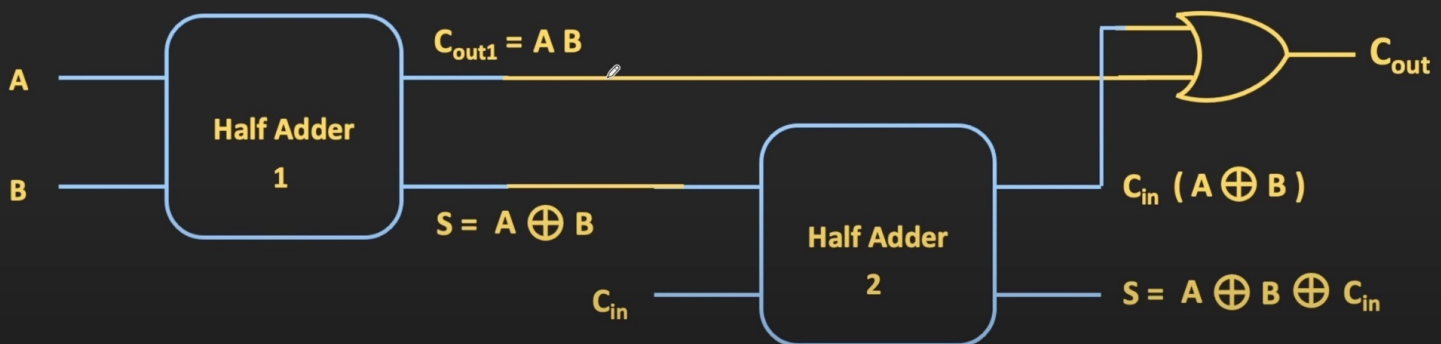
$$C_o = A \cdot B + C_i (A \oplus B)$$



Full Adder using Half Adders

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + C_{in} (A \oplus B)$$



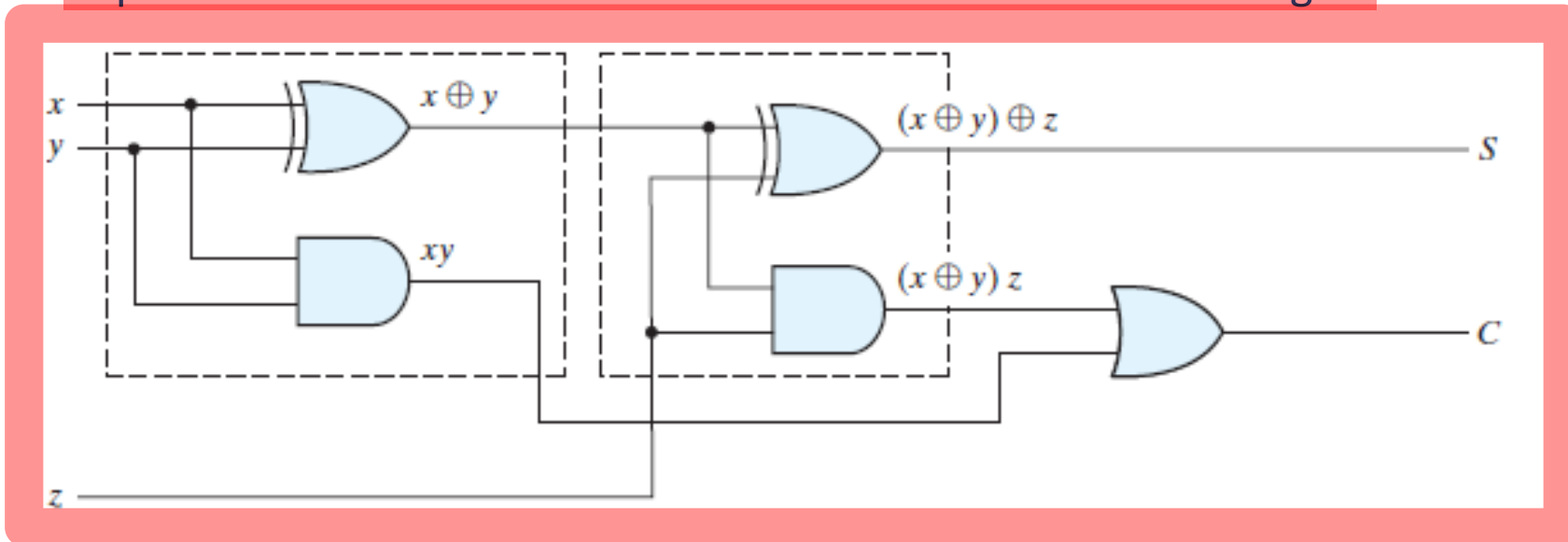
Implement Full adder using two Half Adder

❖ Full Adder Boolean Expression:

$$S = (x \oplus y) \oplus z$$

$$C = (x \oplus y).z + x.y$$

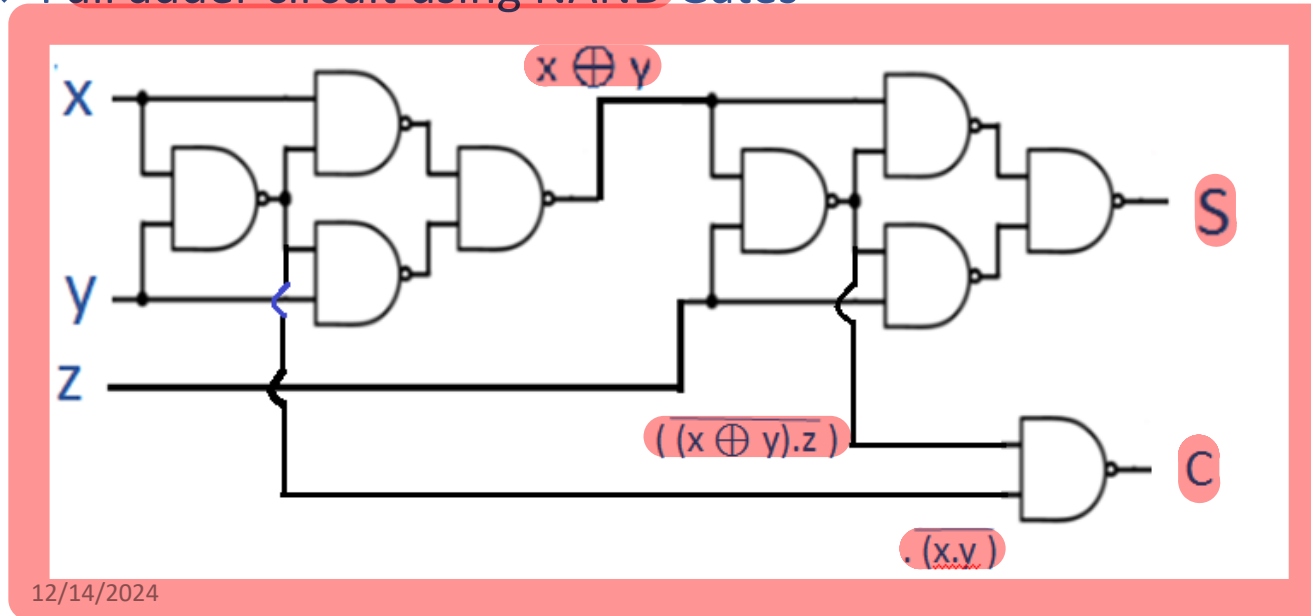
❖ Implementation of full adder with two half adders and an OR gate



Implement Full adder using NAND only

- ❖ Boolean Expression: $S = (x \oplus y) \oplus z$
 $C = (x \oplus y).z + x.y$

- ❖ Full adder circuit using NAND Gates



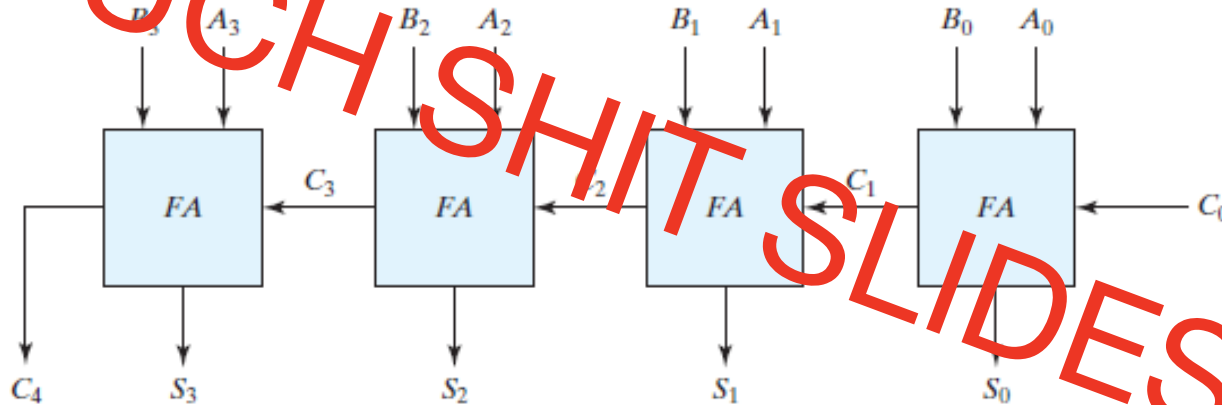
$$C = ((x \oplus y).z + x.y)$$

$$C = ((x \oplus y).z) . (x.y)$$

Combinational Logic Circuits: Half Adder and Full adder

❖ Four-bit adder

Using four Full adder Ripple adder circuit is constructed.

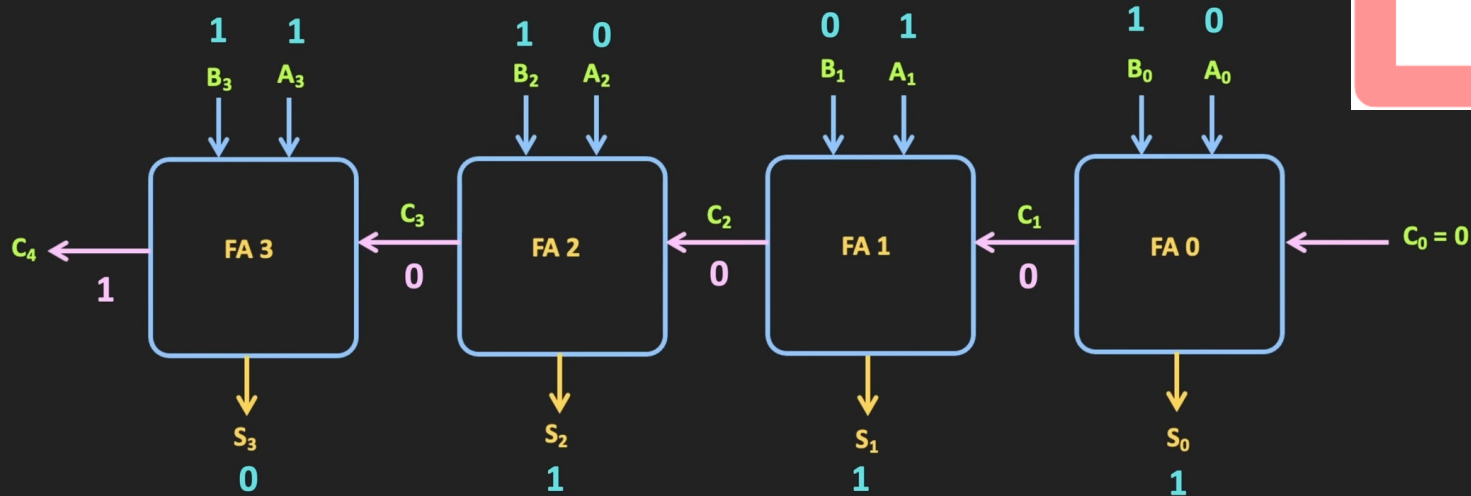


❖ Example

$$\begin{array}{r} 1110 \\ A = 1101 \\ + B = 0111 \\ \hline 10100 \end{array}$$

$$\begin{array}{r}
 1110 \\
 A = 1101 \\
 + B = 0111 \\
 \hline
 10100
 \end{array}$$

4-bit Adder



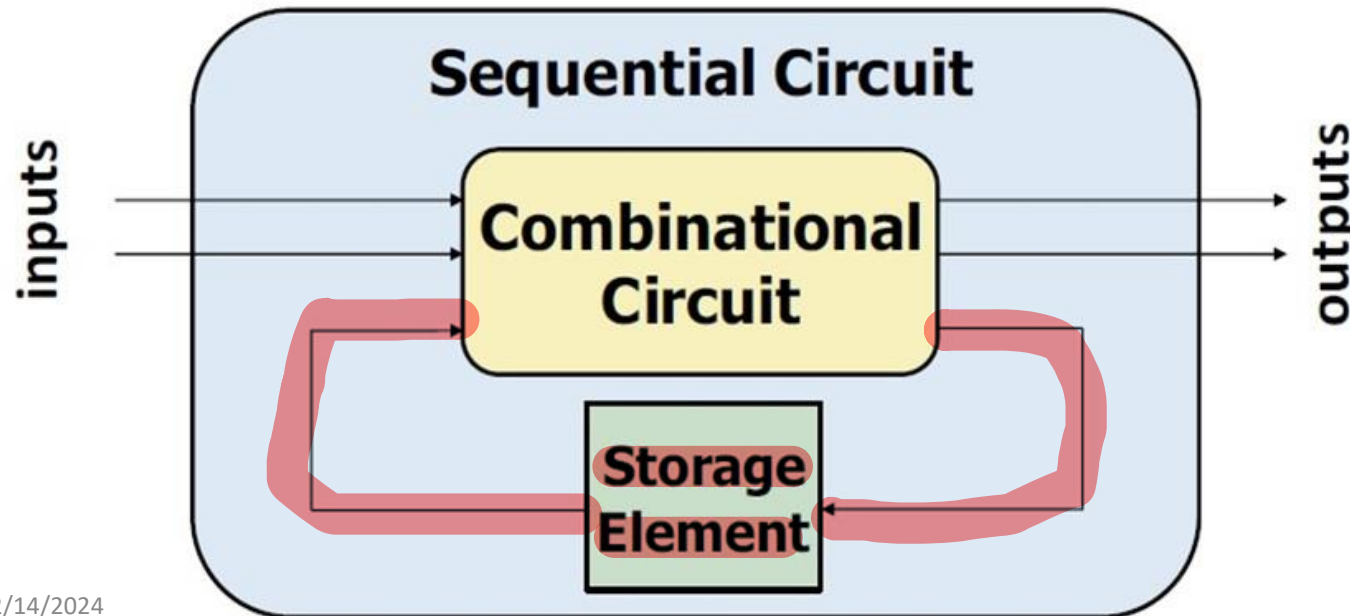
$$\begin{array}{r}
 A_3 \ A_2 \ A_1 \ A_0 \\
 + \ B_3 \ B_2 \ B_1 \ B_0 \\
 \hline
 S_3 \ S_2 \ S_1 \ S_0
 \end{array}$$

$$\begin{array}{r}
 1010 \\
 + 1101 \\
 \hline
 10111
 \end{array}$$

[https://
www.youtube.com/
playlist?
list=PLwjK_iyK4LLCC
pnnybEztvRqxpMyfgar
S](https://www.youtube.com/playlist?list=PLwjK_iyK4LLCCpnnybEztvRqxpMyfgarS)

Sequential Circuits

- Combinational Circuit output depend **only** on **present** input.
- We want circuits that produce the output depending on the **current** and **past** input values - Circuits with **memory**.
- How do we design such a circuit that **stores information**?



- Sequential Circuits are of **two** types

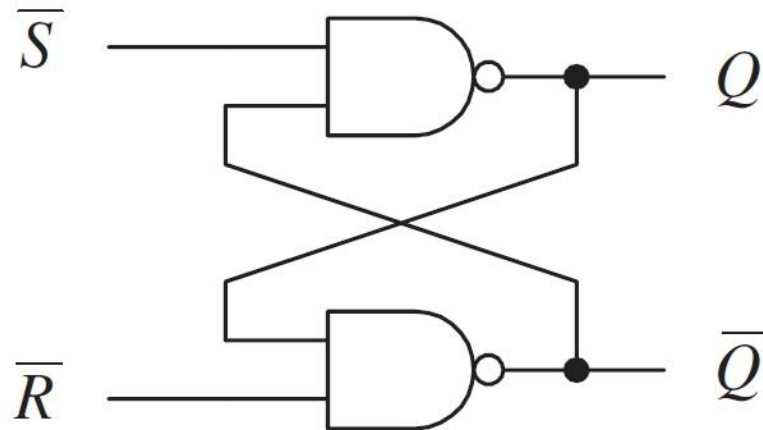
1. Synchronous Circuits:

- ☐ In synchronous sequential circuits, the state of the device changes only at discrete times in response to a **clock pulse**.

2. Asynchronous Circuits:

- ☐ Asynchronous circuit is **not synchronized** by a clock signal; the outputs of the circuit change directly in response to the change in the inputs.

Circuit Diagram



Function Table

INPUTS		OUTPUTS		Status of S'R' Latch
S'	R'	Q	Q'	
0	0	1	1	FORBIDDEN
0	1	1	0	SET
1	0	0	1	RESET
1	1	Q_1	Q'_{-1}	PREVIOUS STATE

1. Set State ($S' = 0, R' = 1$):

- $S' = 0$: Output $Q = 1, \bar{Q} = 0$.
- The latch stores "1" (Set).

2. Reset State ($S' = 1, R' = 0$):

- $R' = 0$: Output $Q = 0, \bar{Q} = 1$.
- The latch stores "0" (Reset).

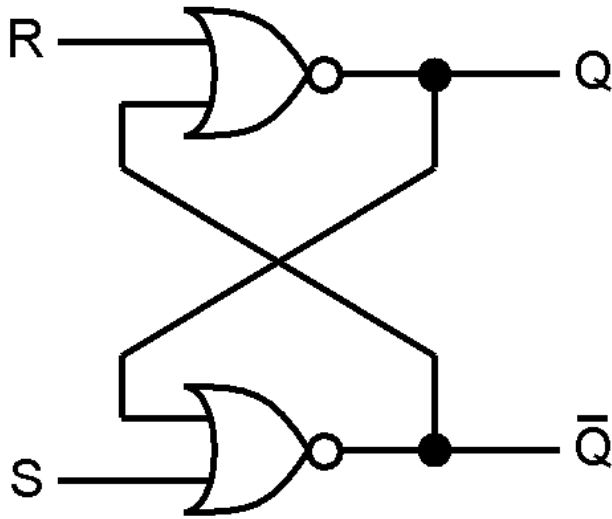
3. Hold State ($S' = 1, R' = 1$):

- No input changes; latch holds its previous state.

4. Invalid State ($S' = 0, R' = 0$):

- Both Q and $\bar{Q}$ become 1, violating the latch condition. Avoid this state.

Circuit Diagram



Function Table

INPUTS		OUTPUTS		Status of RS Latch
R	S	Q	Q'	
0	0	Q_1	Q'_{-1}	PREVIOUS STATE
0	1	1	0	SET
1	0	0	1	RESET
1	1	0	0	FORBIDDEN

1. Set State ($S = 1, R = 0$):

- Input $S = 1$ forces $Q = 1$ and $\bar{Q} = 0$.
- This stores a "1" in the latch.

2. Reset State ($S = 0, R = 1$):

- Input $R = 1$ forces $Q = 0$ and $\bar{Q} = 1$.
- This stores a "0" in the latch.

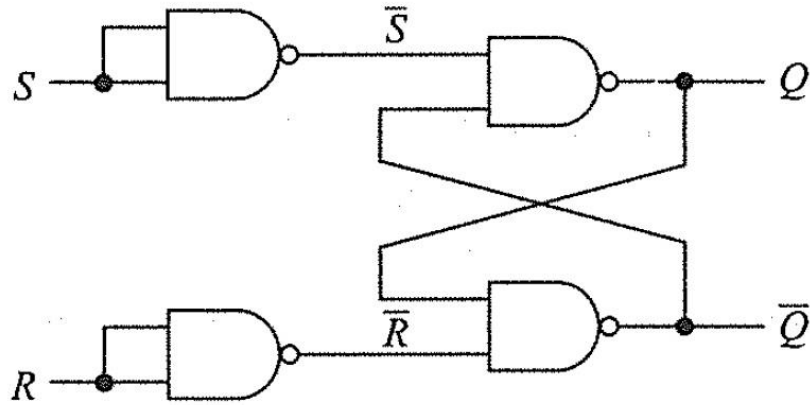
3. Hold State ($S = 0, R = 0$):

- Neither input is active, so the latch maintains its previous state (remembers the last output).

4. Invalid State ($S = 1, R = 1$):

- Both inputs are active, causing Q and $\bar{Q}$ to be undefined (violates the rule $Q \neq \bar{Q}$).

Circuit Diagram



Function Table

INPUTS		OUTPUTS		Status of SR Latch
S	R	Q	Q'	
0	0	Q_1	Q'_{-1}	PREVIOUS STATE
0	1	0	1	RESET
1	0	1	0	SET
1	1	1	1	FORBIDDEN

1. Set State ($S' = 0, R' = 1$):

- The S' input activates the set condition, forcing $Q = 1$ and $\bar{Q} = 0$.

2. Reset State ($S' = 1, R' = 0$):

- The R' input activates the reset condition, forcing $Q = 0$ and $\bar{Q} = 1$.

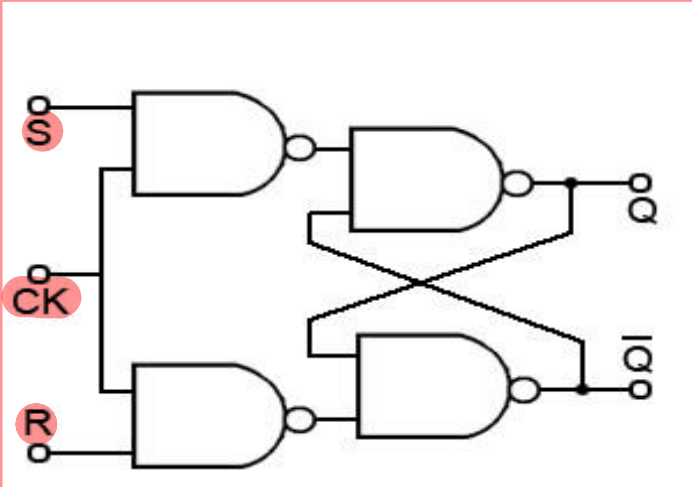
3. Hold State ($S' = 1, R' = 1$):

- Neither input is active, so the latch maintains its previous state.

4. Invalid State ($S' = 0, R' = 0$):

- Both inputs are active, causing an undefined state.

Logic Diagram



Function Table

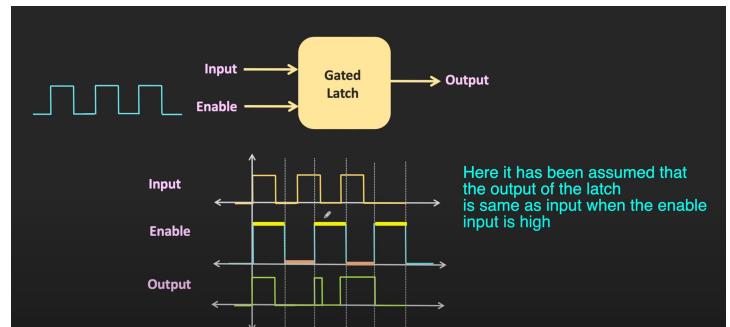
INPUTS			OUTPUTS		Status of SR Flip-Flop
CK	S	R	Q	Q'	
0	X	X	Q_1	Q'_{-1}	PREVIOUS STATE
1	0	0	Q_1	Q'_{-1}	PREVIOUS STATE
1	0	1	0	1	RESET
1	1	0	1	0	SET
1	1	1	1	1	FORBIDDEN

Latch

A **latch** is a basic memory device used in digital electronics. It is a bistable multivibrator, meaning it has two stable states (high and low) and can store a single bit of data. Latches are **level-triggered**, meaning their output changes as soon as the input changes while the enable signal (if present) is active.

Key Characteristics of Latches:

1. **Asynchronous Behavior:** The output depends directly on the inputs when enabled.
2. **Level-Triggered:** Sensitive to the logic level of the enable signal, not to its edge.
3. **Simple Design:** Built using logic gates like NAND or NOR.
4. **Types:** Common types include SR (Set-Reset) latch, D (Data) latch, and JK latch.

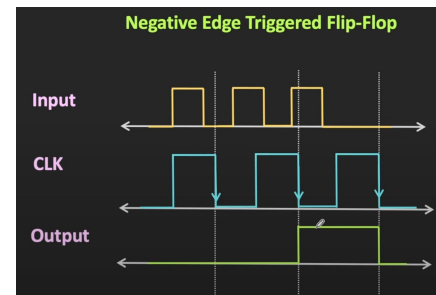
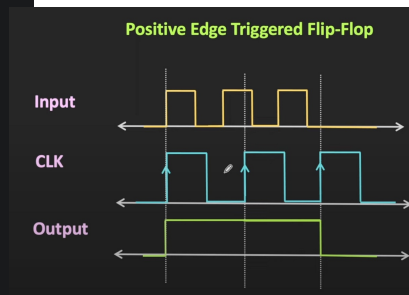
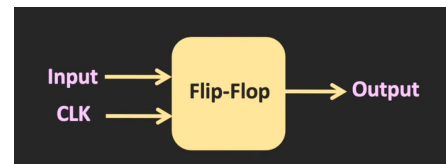


Flip-Flop

A **flip-flop** is also a bistable device used for storing a bit of data, but it is **edge-triggered**, meaning it changes its state only on the rising or falling edge of a clock signal. Flip-flops are used in synchronous circuits and serve as the basic building blocks of registers, counters, and memory elements.

Key Characteristics of Flip-Flops:

1. **Synchronous Behavior:** Controlled by a clock signal.
2. **Edge-Triggered:** State changes occur only on clock edges (positive or negative edge).
3. **More Complex:** Includes additional control inputs like enable or preset/clear.
4. **Types:** Common types include SR flip-flop, D flip-flop, JK flip-flop, and T (Toggle) flip-flop.



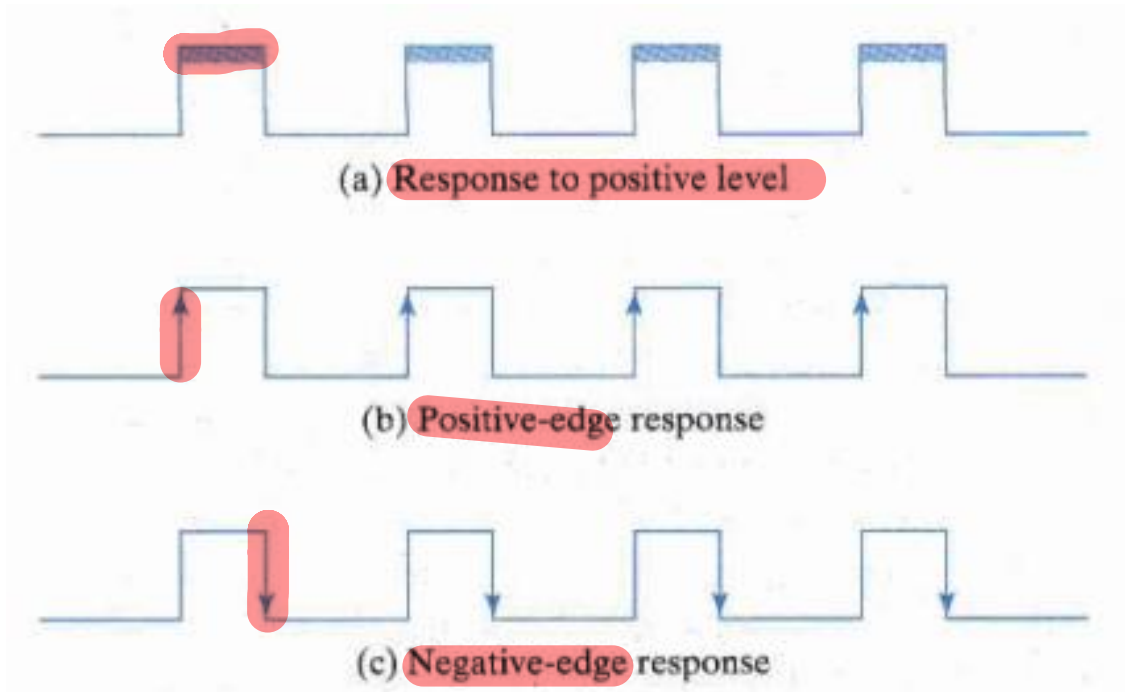
Comparison: Latch vs. Flip-Flop

Feature	Latch	Flip-Flop
Triggering	Level-triggered	Edge-triggered
Clock Dependency	May or may not have it	Always clock-dependent
Usage	Asynchronous circuits	Synchronous circuits
Design Complexity	Simpler	More complex

1. Flip – flops are the **building blocks** of sequential circuits.
But these are built from **latches**.
2. Flip- flops **continuously checks** its inputs and changes its outputs
correspondingly only at time instant determined by the **clock signal**.
3. Flip – flops is **sensitive** to signal change. They transfer data only at single time instant and data can't be changed till next signal change.
➤ Therefore they are used as **registers**.
4. It is an **edge triggered circuit**, means that the output and the next state input changes, when there's a change in the clock pulse whether it can may be **POSITIVE (/up)** or **NEGATIVE (/**

Comparison between Flip-flop and Latch

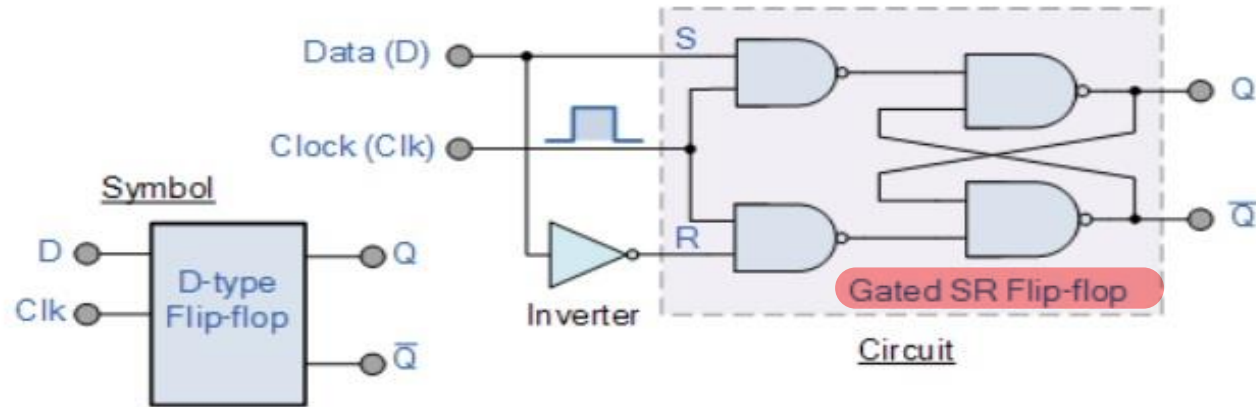
Flip-Flop	Latch
Flip-flop utilizes an edge triggering approach.	Latch follows a level triggering approach.
The clock signal is present.	The clock signal is absent.
You can design it using Latches along with a clock.	You can design it using Logic gates.
Flip-flop is sensitive to the applied input and the clock signal.	Latches are sensitive to the applied input signal- only when enabled.
It has a slow operating speed.	It has comparatively fast operating speed.
You can classify a flip-flop into a synchronous or asynchronous flip-flop.	A user cannot classify the Latch this way.
A flip-flop checks the inputs. It only changes the output at times defined by any control signal like the clock signal.	The latch responds to the changes in inputs continuously as soon as it checks the inputs.
A flip-flop is synchronous. It works based on the clock signal.	A latch is asynchronous. It does not work based on the time signal.



Flip-Flops will change the Output when the clock remains as logic 1 as shown in figure (a)

Flip-Flops will change the Output when the clock changes from logic 0 to Logic 1 as shown in figure (b)

Flip-Flops will change the Output when the clock changes from logic 1 to Logic 0 as shown in figure (c)



□ The **D Flip Flop** is by far the most important of the clocked flip-flops as it ensures that inputs **S** and **R** are **never equal** to one at the same time.

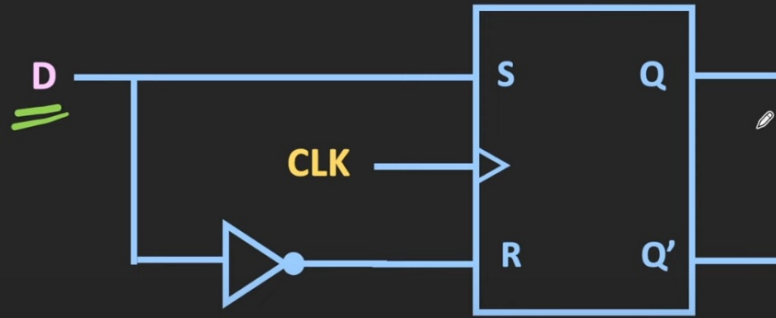
□ The **D Flip Flop** are constructed from a **SR flip-flop** with an **inverter** added between the **S** and the **R** inputs to allow for a single D (Data) input.

Electronic Principles and Devices

D Flip – Flop (Characteristic Table)

Truth Table

CLK	D	Q_n	Q_{n+1}
0	X	0 1	0 1
↑	0	0/1	0
↑	1	0/1	1

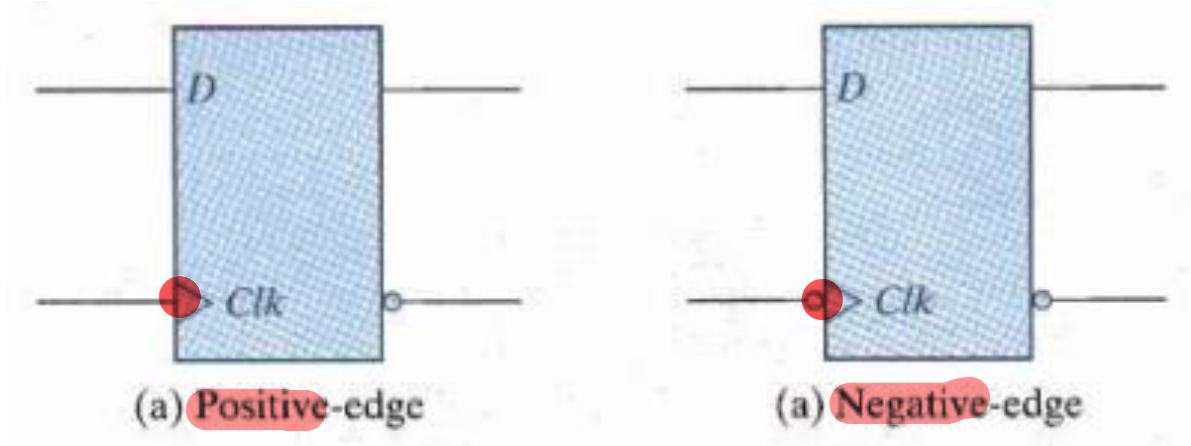


CLK	D	S	R	Q_n	Q_{n+1}
0	X	X	X	0 1	0 1
↑	0	0	1	0/1	0
↑	1	1	0	0/1	1

❑ The D Flip-flop will store and output whatever logic level is applied to its **DATA** input.

❑ Once the clock input goes LOW the “set” and “reset” inputs of the flip-flop are both held at logic level “1”, so it will not change its state.

❑ ↓ and ↑ indicates direction of clock pulse as it is assumed D-type flip flops are edge triggered.



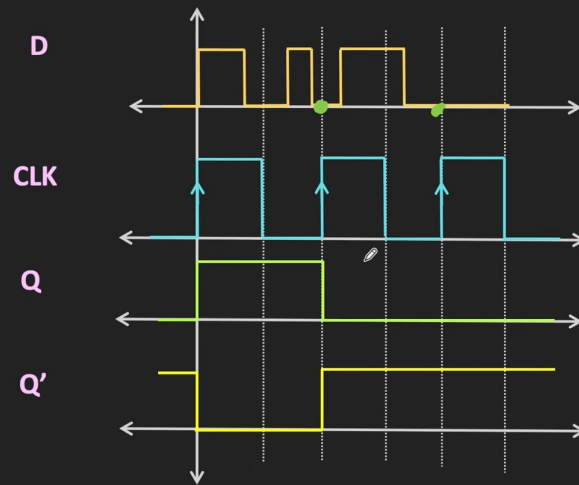
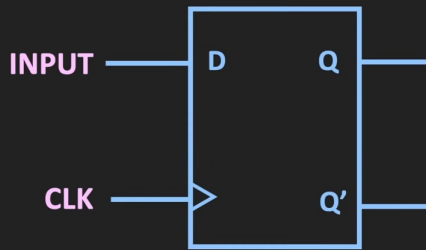
In D Flip-Flops, **Absence** of the **bubble** in front of the **Clk** indicates **Positive edge triggered** as shown in figure (a)

12/14/2024

In D Flip-Flops, **Presence** of the **bubble** in front of the **Clk** indicates ~~Positive~~ ^{negative} edge triggered as shown in figure (b)

Positive Edge Triggered D Flip-Flop

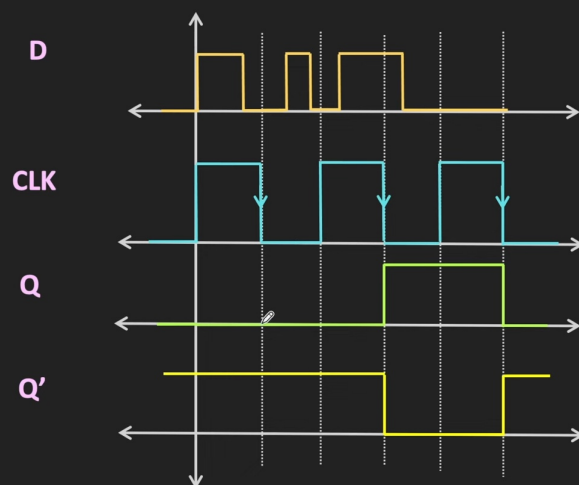
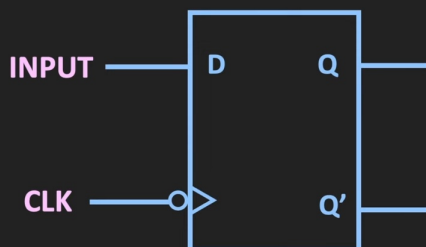
Symbol



ALL ABOUT ELECTRONICS

Negative Edge Triggered D Flip-Flop

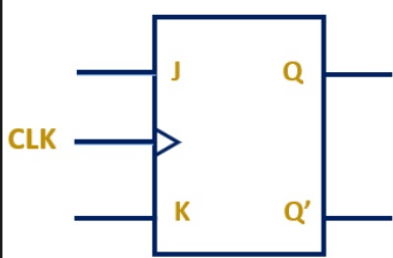
Symbol



ALL ABOUT ELECTRONICS

In the **SR Flip-Flop**, when both inputs S and R are 1 then the output of the flip-flop is indeterminate. That issue can be resolved using the JK Flip-Flop. Similar to the SR Flip-Flop, the JK flip-flop has two inputs. And using the two inputs the flip-flop can be set, reset, hold (memory) or toggled. Unlike the SR flip-flop, in the JK flip-flop, when both inputs J and K are 1 then the output of the flip-flop toggles. The symbol of edge triggered JK flip-flop and its truth table is shown below.

Symbol



Truth Table

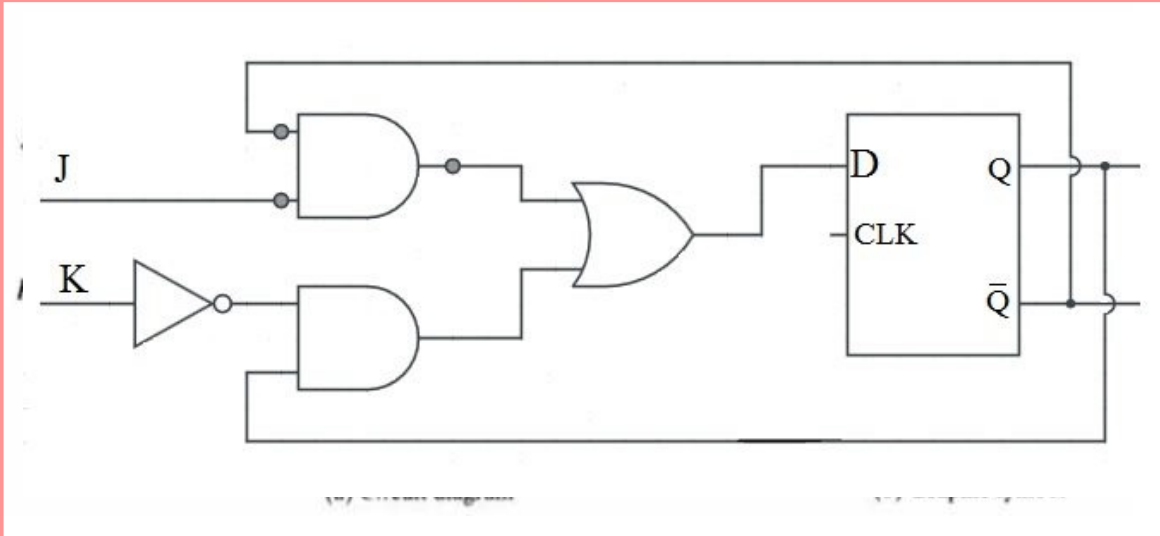
CLK	J	K	Q_{n+1}
↑	0	0	Q_n
↑	0	1	0
↑	1	0	1
↑	1	1	Q_n'

In the JK flip-flop, at the rising edge of the clock, when **J = 0 and K = 0** then flip-flop **retains (holds) the current state**. When **J = 0 and K = 1**, then flop-flop **resets to 0**. When **J = 1 and K = 0**, then flip-flop **sets the output to 1**. And when **J = 1 and K = 1** then **output of the flip-flop toggles**. When the clock signal is low, then irrespective of the value of J and K inputs, the flop-flop retains the present state. The detailed truth table with all different possibilities of Q_n , J and K inputs are shown below, where Q_n is the present state and Q_{n+1} is the next state of the flip-flop.

CLK	J	K	Q_n	Q_{n+1}	Q_{n+1}
0	X	X	0/1	0/1	Q_n
↑	0 0	0 0	0 1	0 1	Q_n
↑	0 0	1 1	0 1	0 0	0
↑	1 1	0 0	0 1	1 1	1
↑	1 1	1 1	0 1	1 0	Q_n'

Electronic Principles and Devices

JK Flip – Flop using D flip flop



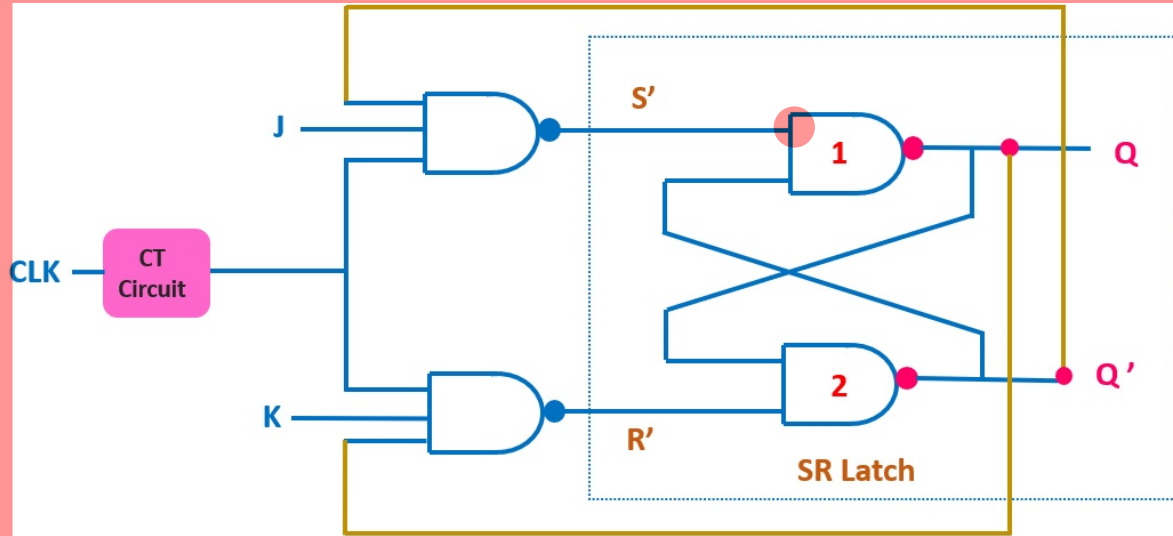
□ The Inputs are **J-K** along with the Clk

➤ J-K inputs comes after its inventors **Jack Kilby**.

INPUTS			OUTPUTS		Status of JK Flip-Flop
CLK	J	K	Q	Q'	
0	X	X	Q_{-1}	Q'_{-1}	PREVIOUS STATE
1	0	0	Q_{-1}	Q'_{-1}	PREVIOUS STATE
1	0	1	0	1	RESET
1	1	0	1	0	SET
1	1	1	Q'_{-1}	Q_{-1}	TOGGLE

Electronic Principles and Devices

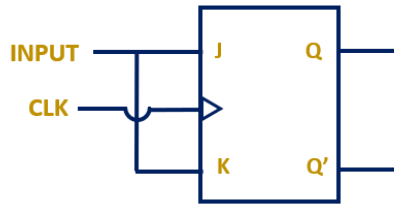
JK Flip – Flop using NAND Gates only



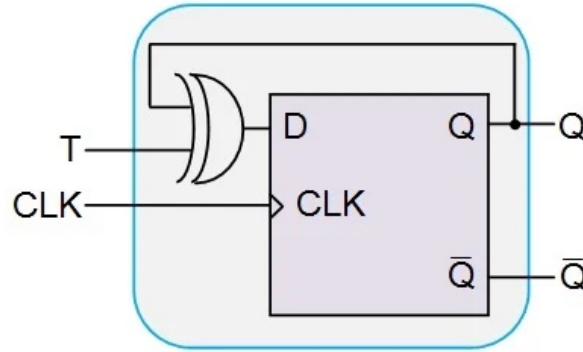
INPUTS			OUTPUTS		Status of JK Flip-Flop
CLK	J	K	Q	Q'	
0	X	X	Q_{-1}	Q'_{-1}	PREVIOUS STATE
1	0	0	Q_{-1}	Q'_{-1}	PREVIOUS STATE
1	0	1	0	1	RESET
1	1	0	1	0	SET
1	1	1	Q'_{-1}	Q_{-1}	TOGGLE

- ❑ The **S** and **R** inputs of SR slip-flops are now replaced by **J-K** inputs.
 - J-K inputs comes after its inventors **Jack Kilby**.
 - Then this equates to: **J = S** and **K = R**.

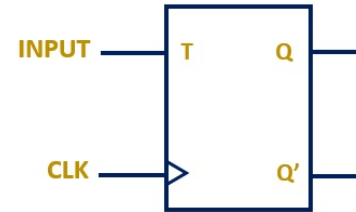
- **JK flip-flop** is a sequential circuit, whose state transitions are **synchronized** with the **clock pulse** (+ve / -ve) edge triggered.
- In **JK flip-flop** all the rows are a valid state in the characteristic table, also when $J=K=1$, the output toggles, **Toggle state**.
- **D flip-flop** is a sequential circuit, whose output is same as the input and hence also called as **transparent flip-flop**.



(a) From JK flip-flop



(b) From D flip-flop



(c) Graphic symbol

T Flip-Flop using JK Flip-Flop & D Flip-Flop

- ❑ T Flip-Flop is a **synchronous** device, where high to low or low to high transitions is passed through **clock signal** which **changes** the output state of Flip-Flop.

INPUTS		OUTPUTS		Status of T Flip-Flop
CLK	T	Q	Q'	
0	X	Q_{-1}	Q'_{-1}	PREVIOUS STATE
1	0	Q_{-1}	Q'_{-1}	PREVIOUS STATE
1	1	Q'_{-1}	Q_{-1}	TOGGLE

Truth Table

CLK	T	Q_{n+1}
↑	0	Q_n
↑	1	Q_n'

Advantages of T flip-flop:

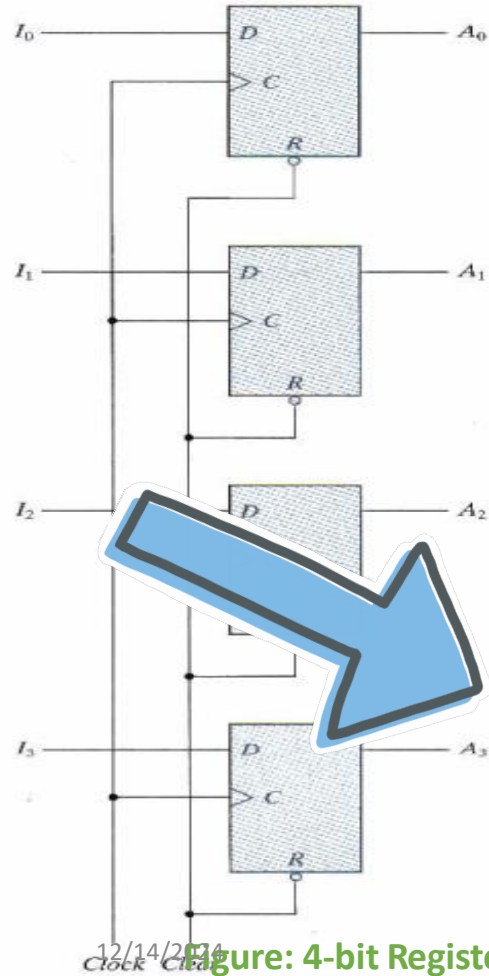
- These Flip-Flops has a **toggle input** and a **clock**.
When a clock is triggered it **inverts** the value of Flip-Flops.

- A **register** is a group of flip-flops, each one of which shares a common clock and is capable of storing one bit of information.
- An **n -bit register** consists of a group of n flip-flops capable of storing n bits of binary information.
- In addition to the flip-flops, a register may have combinational gates that perform certain data-processing tasks.
- In its broadest definition, a register consists of a group of flip-flops together with gates that affect their operation.
- The flip-flops hold the binary information, and the gates determine how the information is transferred into the register.

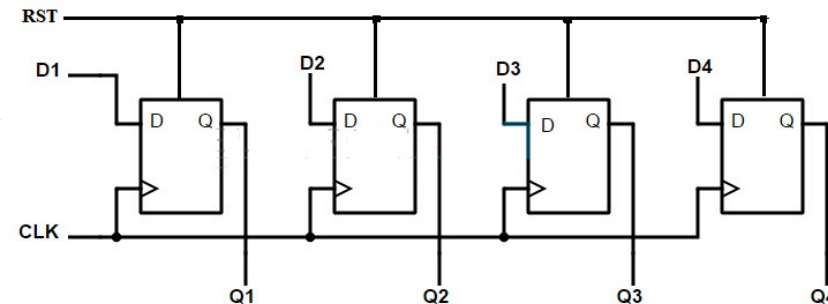
Electronic Principles and Devices

4-bit Register:

- Various types of registers are available commercially.
- The simplest register is one that consists of only flip-flops, without any gates.
- Following Figure (next slide) shows such a register constructed with four D-type flip-flops to form a four-bit data storage register.
- The common clock input triggers all flip-flops on the positive edge of each pulse, and the binary data available at the four inputs are transferred into the register.
- The four outputs can be sampled at any time to obtain the binary information stored in the register.
- The input Clear goes to the active-low R (reset) input of all four flip-flops. When this input goes to 0, all flip-flops are reset asynchronously.
- The R inputs must be maintained at logic 1 (i.e., de-asserted) during normal clocked operation.



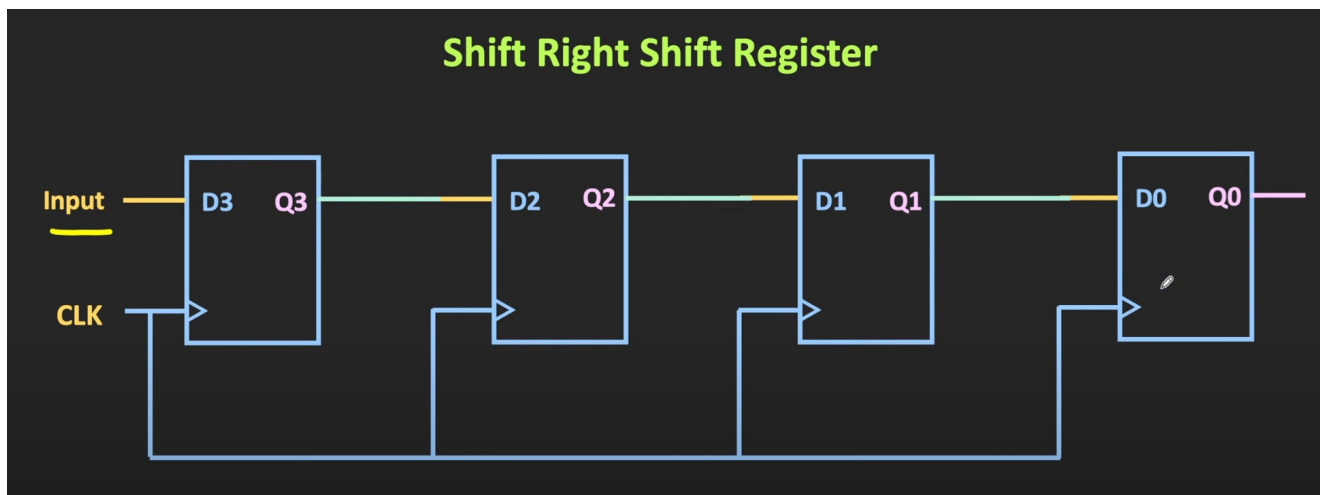
- The **transfer** of new information into a register is referred to as **loading or updating** the register.
- If **all the bits** of the register are loaded **simultaneously** with a common clock pulse, we say that the **loading is done in parallel**.
- A **clock edge applied to the C inputs** of the register of Figure, will load all four inputs in parallel.
- In this configuration, **if the contents of the register must be left unchanged, the inputs must be held constant or the clock must be inhibited from the circuit.**

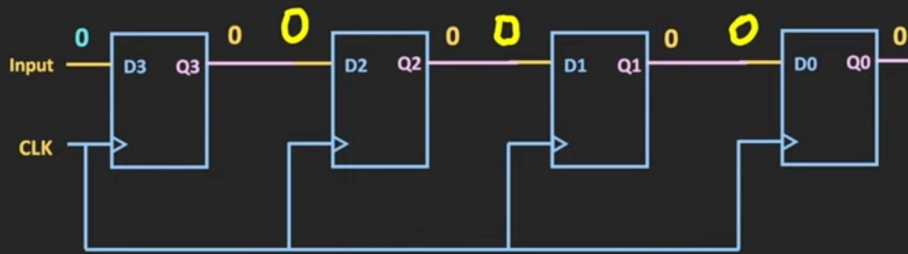


12/14/2016 **Figure: 4-bit Register**

- A register capable of shifting the binary information held in each cell to its neighboring cell, in a selected direction, is called a *shift register*.
- The logical configuration of a shift register consists of a chain of flip-flops in cascade, with the output of one flip-flop connected to the input of the next flip-flop.
- All flip-flops receive common clock pulses, which activate the shift of data from one stage to the next.
- There are four different types of shift registers:
 - Serial In Serial Out (SISO)
 - Serial In Parallel Out (SIPO)
 - Parallel In Serial Out (PISO)
 - Parallel In Parallel Out (PIPO)

- The simplest possible shift register (SISO) is one that uses only flip-flops, as shown in the following Figure .
- The output of a given flip-flop is connected to the D input of the flip-flop at its right.
- This shift register is unidirectional (left-to-right).
- Each clock pulse shifts the contents of the register one bit position to the right.





MSB LSB
1 0 1 0
=

CLK	Input	Q3	Q2	Q1	Q0
0	X	0	0	0	0
↑	0	0	0	0	0

CLK	Input	Q3	Q2	Q1	Q0
0	X	0	0	0	0
↑	0	0	0	0	0
↑	1	1	0	0	0

MSB LSB
1 0 1 0

CLK	Input	Q3	Q2	Q1	Q0
0	X	0	0	0	0
↑	0	0	0	0	0
↑	1	1	0	0	0
↑	0	0	1	0	0

MSB LSB
1 0 1 0

4 CLK Cycles

CLK	Input	Q3	Q2	Q1	Q0
0	X	0	0	0	0
↑	0	0	0	0	0
↑	1	1	0	0	0
↑	0	0	1	0	0
↑	1	1	0	1	0

MSB LSB
1 0 1 0

CLK	Input	Q ₃	Q ₂	Q ₁	Q ₀
0	X	0	0	0	0
↑	0	0	0	0	0
↑	1	1	0	0	0
↑	0	0	1	0	0
↑	1	1	0	1	0



MSB LSB
1 0 1 0

INPUT

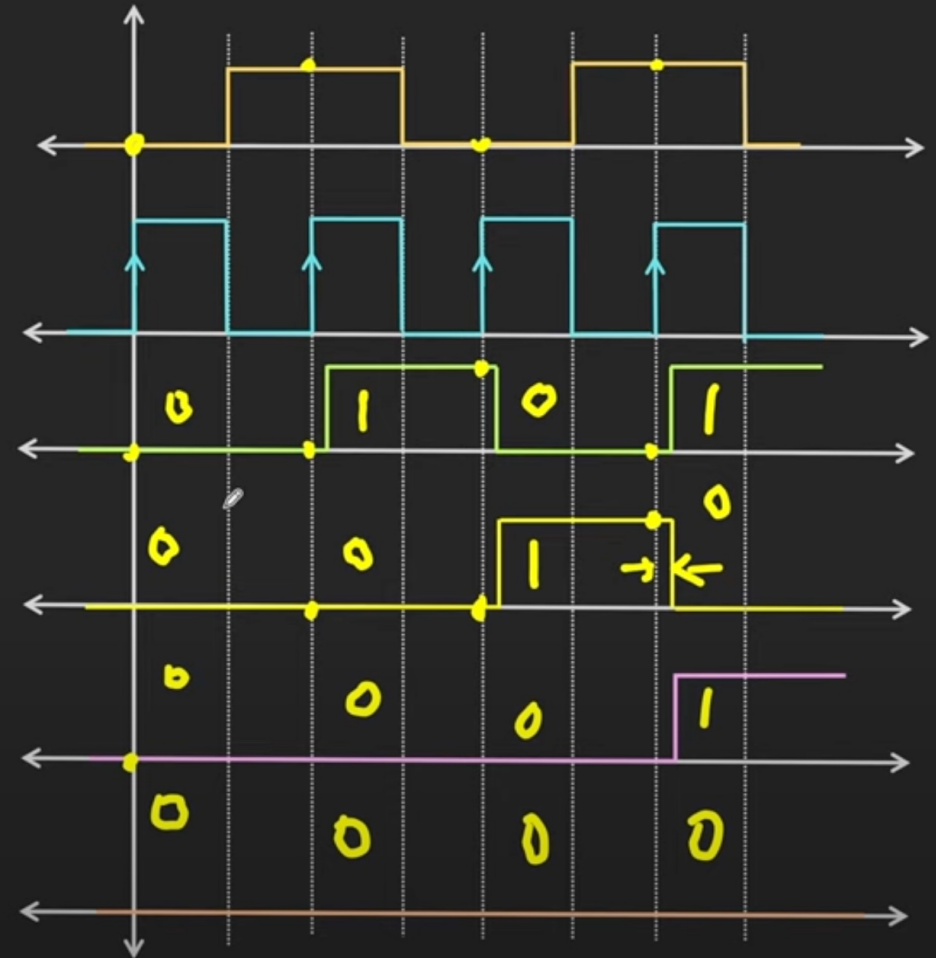
CLK

Q₃

Q₂

Q₁

Q₀



Timing Diagram for 4-bit Register

Electronic Principles and Devices

4-bit Serial Input Serial Output (SISO) register:

Clock Pulse No	QA	QB	QC	QD
0	0	0	0	0
1	1	0	0	0
2	0	1	0	0
3	0	0	1	0
4	0	0	0	1
5	0	0	0	0

Data movement through a shift register

Electronic Principles and Devices

4-bit Serial Input Serial Output (SISO) register:

Example:

The register is shifted six times to the right with the serial input being 0110. What is the content of the register after each shift?

Answer:

0110; 0011; 1001; 1100; 0110; 0011.

Electronic Principles and Devices

4-bit Serial Input Serial Output (SISO) register:

Example:

The contents of a four-bit register is initially 0110. The register is shifted six times to the right with the serial input being 1011100.

What is the content of the register after each shift?

Answer:

0110; 0011; 0001; 1000; 1100; 1110; 0111; 1011

	Data	QA	QB	QC	QD
	initial	0	1	1	0
1 st	0	0	0	1	1
2 nd	0	0	0	0	1
3 rd	1	1	0	0	0
4 th	1	1	1	0	0
5 th	1	1	1	1	0
6 th	0	0	1	1	1
7 th	1	1	0	1	1

- A **register (group of flip-flops)** that goes through a prescribed sequence of states upon the application of input pulses is called a *counter*.
- The input pulses may be **clock pulses**, or they may originate from some external source and may occur at a fixed interval of time or at random.
- The sequence of states may follow the binary number sequence or any other sequence of states.
- A counter that follows the binary number sequence is called a *binary counter*.
- An **n-bit binary counter** consists of n flip-flops and can count in binary from 0 through $2^n - 1$.

- Counters are available in **two categories**: ripple (or asynchronous) counters and synchronous counters.
- In a **ripple counter**, a flip-flop output transition serves as a source for triggering other flip-flops.
- In other words, the **C input** of some or all flip-flops are triggered, **not by the common clock pulses**, but rather by the transition that occurs in other flip-flop outputs.
- In a **synchronous counter**, the **C inputs** of all flip-flops receive the **common clock**.

A **counter** is a sequential circuit made up of flip-flops that progresses through a specific sequence of states upon receiving input pulses. These circuits are widely used in digital electronics to count occurrences of events, generate timing signals, or divide frequencies.

1. Ripple Counter (Asynchronous Counter)

- **How it works:**
 - In a ripple counter, only the first flip-flop receives the clock signal.
 - The output of each flip-flop serves as the clock input for the next flip-flop.

2. Synchronous Counter

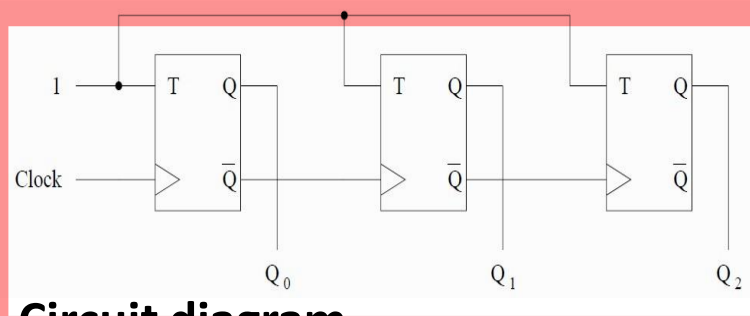
- **How it works:**
 - All flip-flops receive the clock pulse simultaneously, eliminating propagation delays.
 - Logic gates control the state transitions based on the current output.

Binary Counters

- Binary counters follow a **binary sequence** where each state represents a binary number.
- An n -bit counter can count from 0 to $2^n - 1$, with n flip-flops.
- **Example:**
 - A 3-bit counter has 8 states: 000, 001, 010, ..., 111 (0 to 7 in decimal).

Electronic Principles and Devices

3 bit Asynchronous up- counter:



Circuit diagram

Truth Table

Number of clock pulses	Q ₂ (MSB)	Q ₁	Q ₀ (LSB)
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

12/14/2024

Timing Diagram

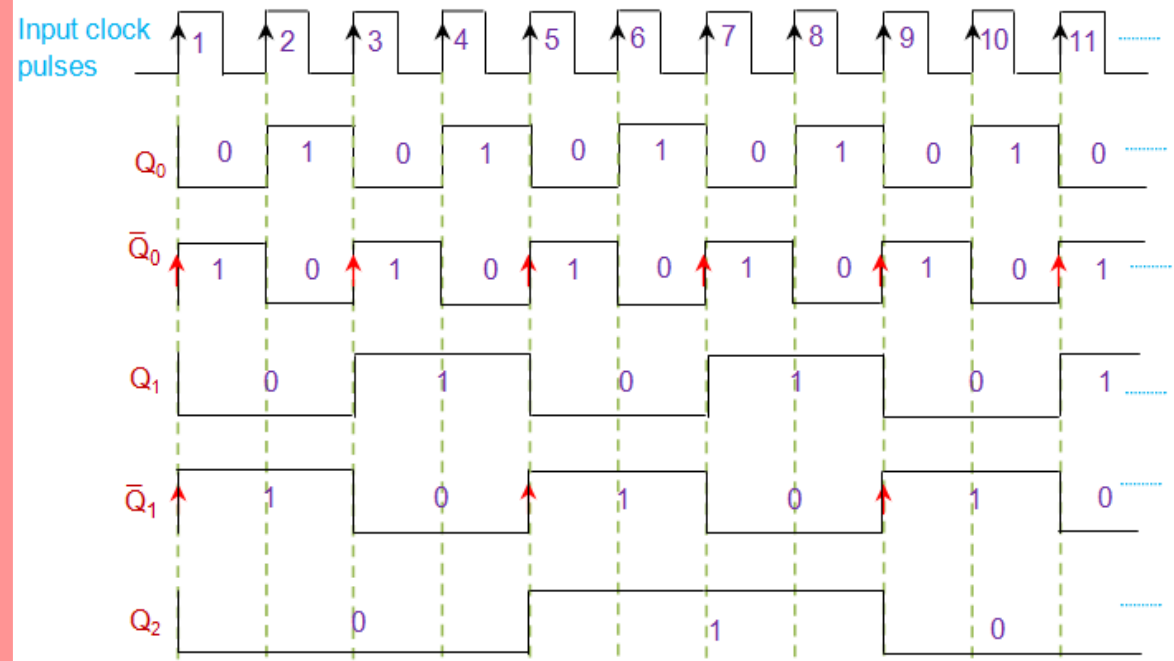


Figure 4 Timing diagram for 3-bit asynchronous positive edge triggered up-counter

3-Bit Asynchronous Up and Down Counter Using T Flip-Flops

A 3-bit asynchronous counter with T flip-flops can operate as either an **up counter** (counts from 0 to 7) or a **down counter** (counts from 7 to 0) depending on how the clock signals are connected.

Overview of T Flip-Flop

- The T flip-flop toggles its state on every clock pulse when $T = 1$.
- If $T = 0$, the output does not change.

Why Are Inverted Outputs Used?

In a standard asynchronous **up counter**:

- Each flip-flop toggles when the output of the previous flip-flop transitions from **1 to 0** (i.e., at the falling edge of its output clock).

In this design:

- The inverted outputs ($\overline{Q_0}$, $\overline{Q_1}$) are used as the clock signals for the next flip-flops.
- This configuration means the subsequent flip-flops toggle on the **rising edge** of the inverted outputs, which corresponds to the **falling edge** of the original outputs (Q_0 , Q_1).

This approach effectively achieves the same result as the standard configuration but relies on inverted outputs to trigger the next flip-flops.

1. Toggle Logic:

- A T flip-flop toggles on the positive edge of its clock input.
- If the inverted output ($\overline{Q}$) is used as the clock signal, the flip-flop toggles when the original output (Q) falls from 1 to 0.
- This matches the behavior required for an up counter, where the next flip-flop toggles when the previous flip-flop completes a cycle.

2. Timing and Sequence:

- The counting sequence remains the same: $000 \rightarrow 001 \rightarrow 010 \rightarrow 011 \rightarrow 100 \rightarrow 101 \rightarrow 110 \rightarrow 111$.

Timing Diagram and Behavior:

Referring to the **timing diagram in the image**:

1. The **input clock pulses** drive the first flip-flop (Q_0).
2. Each subsequent flip-flop (Q_1 , Q_2) toggles on the positive edge of the inverted clock signal derived from the previous flip-flop ($\overline{Q_0}$ for Q_1 and $\overline{Q_1}$ for Q_2).
3. This setup ensures:
 - Q_0 toggles on every input clock pulse.
 - Q_1 toggles when Q_0 completes a full cycle ($0 \rightarrow 1 \rightarrow 0$).
 - Q_2 toggles when Q_1 completes a full cycle.

Flip-Flop 0 (Q_0 , LSB):

- Directly connected to the input clock signal.
- Toggles its state on every positive edge of the clock.

Flip-Flop 1 (Q_1 , Middle Bit):

- Triggered by the positive edge of Q_0 .
- Toggles its state when Q_0 transitions from 0 to 1, effectively dividing the clock frequency by 2.

Flip-Flop 2 (Q_2 , MSB):

- Triggered by the positive edge of Q_1 .
- Toggles its state when Q_1 transitions from 0 to 1, dividing the clock frequency by 4.

Counting Sequence:

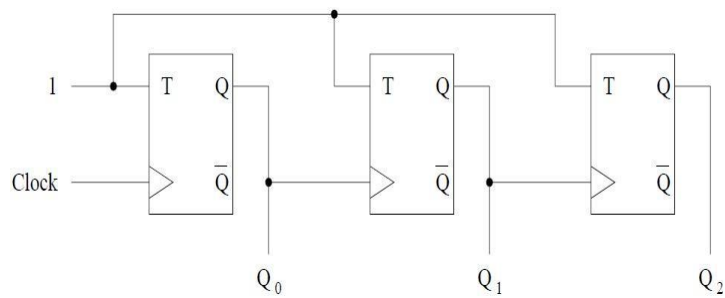
- The counter follows this binary sequence: $111 \rightarrow 110 \rightarrow 101 \rightarrow 100 \rightarrow 011 \rightarrow 010 \rightarrow 001 \rightarrow 000$.
- Each flip-flop toggles at half the frequency of the previous flip-flop, generating the downward counting sequence.

In a standard **down counter**:

- Each flip-flop toggles on the **positive edge** of the clock signal from the previous flip-flop's output.
- The non-inverted outputs (Q_0 , Q_1) are used to ensure that the higher-order flip-flops toggle when the lower-order flip-flops transition from **0 to 1**.

Electronic Principles and Devices

3 bit Asynchronous down - counter:



Circuit diagram

Truth Table

Number of clock pulses	Q ₂ (MSB)	Q ₁	Q ₀ (LSB)
0	0	0	0
1	1	1	1
2	1	1	0
3	1	0	1
4	1	0	0
5	0	1	1
6	0	1	0
7	0	0	1

Timing Diagram

